UNIVERSITÄT
LEIPZIG

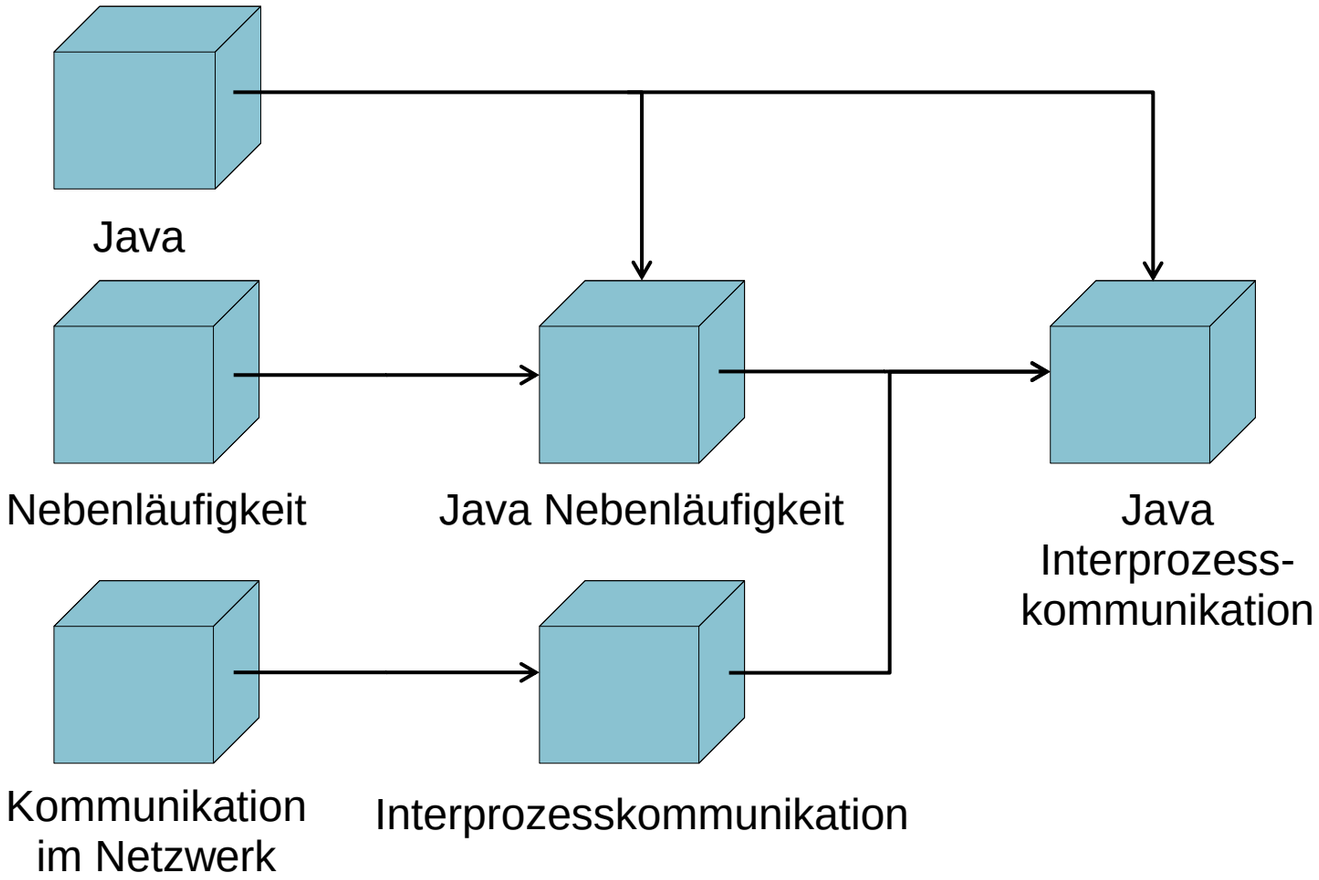
Grundlagen verteilter Systeme

Entwicklung verteilter Anwendungen

Leipzig, 09.06.2022

Dr. Pascal Kovacs

AGENDA

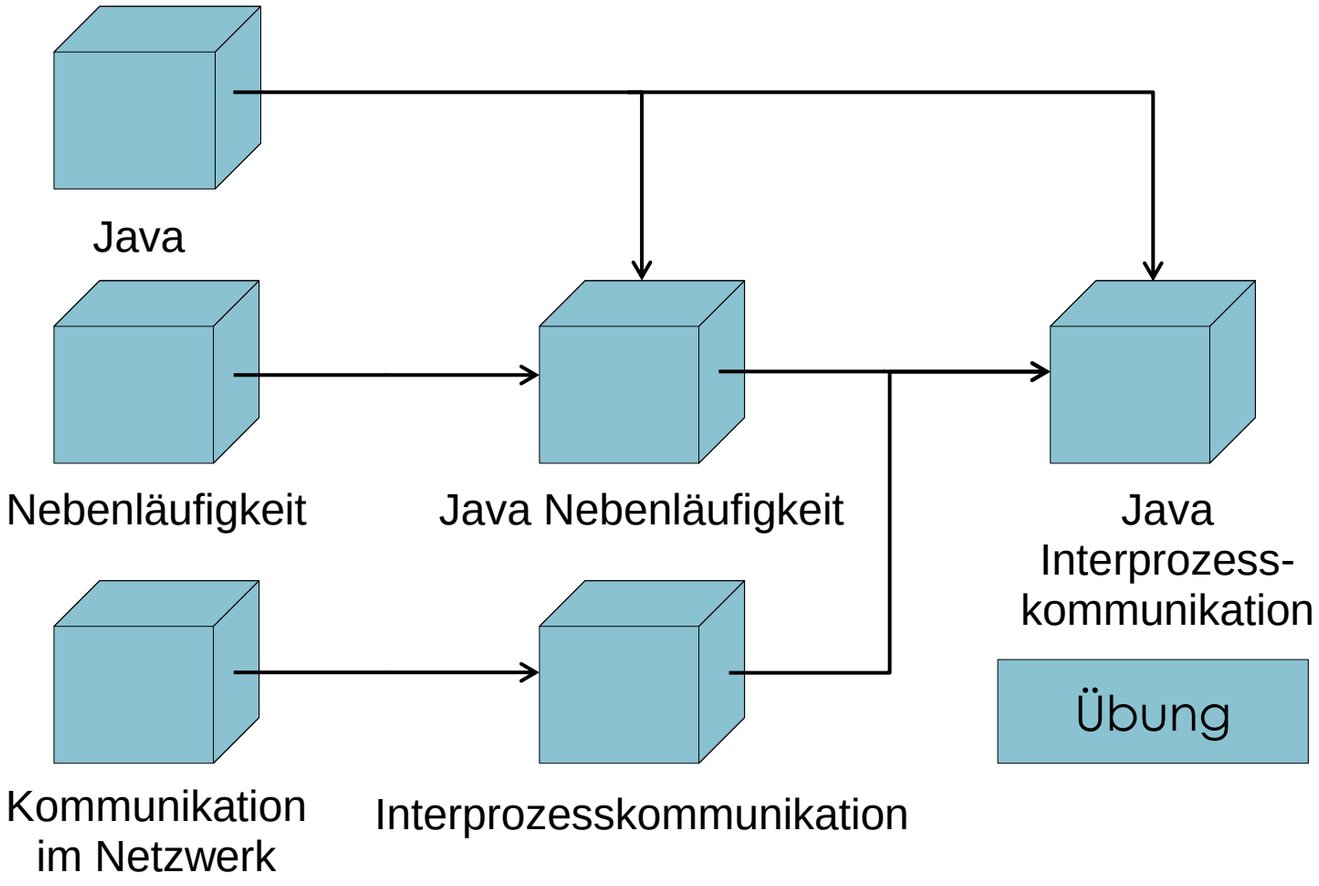

Java
Nebenläufigkeit
Java Nebenläufigkeit
Java Interprozess-kommunikation
Kommunikation im Netzwerk
Interprozesskommunikation

ZIELE

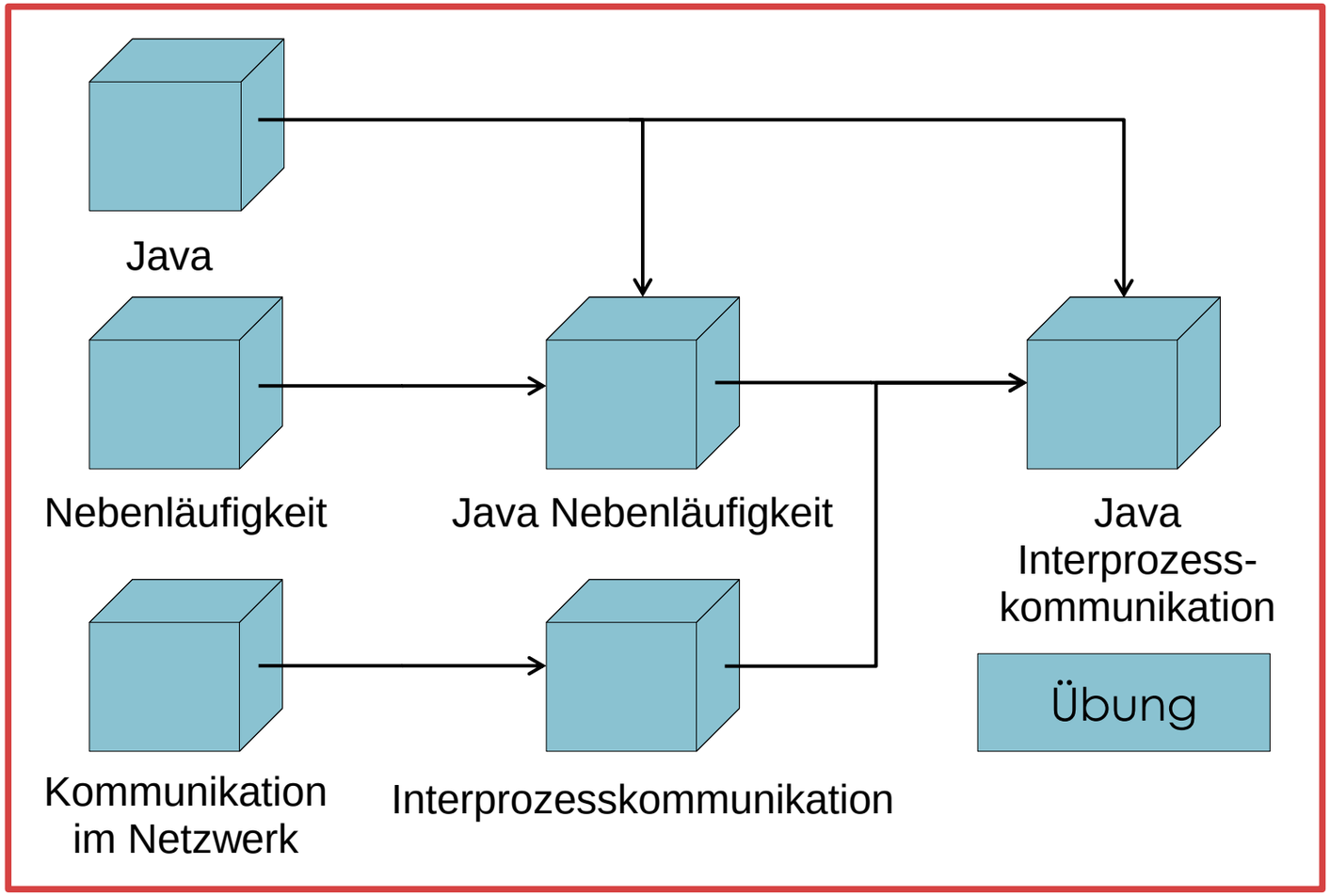
- Sockets in Java
- Java RMI

BEISPIELFRAGE

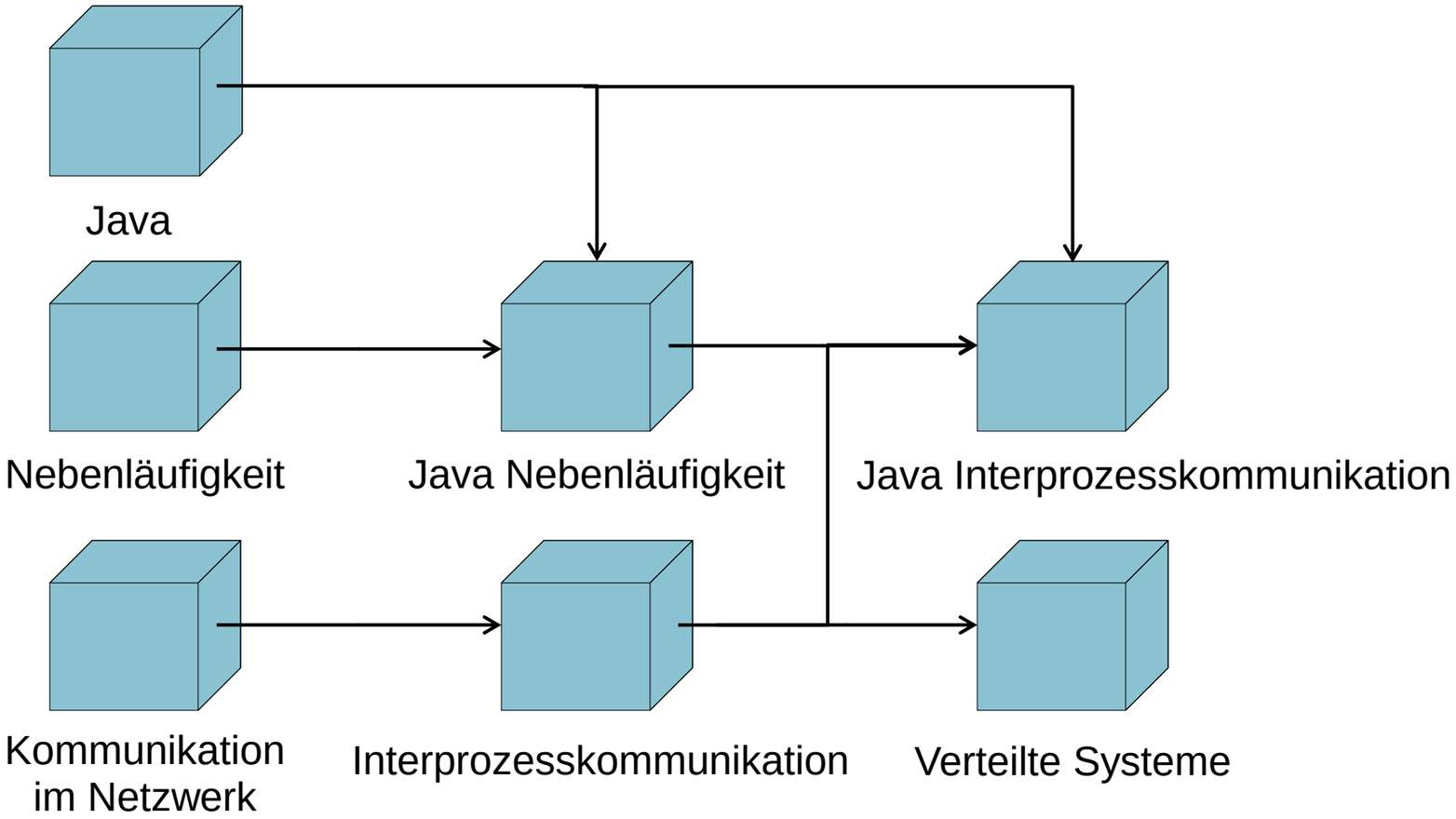
AGENDA


Java
Nebenläufigkeit
Java Nebenläufigkeit
Java Interprozess-kommunikation
Übung
Kommunikation im Netzwerk
Interprozesskommunikation

AGENDA


Java
Nebenläufigkeit
Java Nebenläufigkeit
Java Interprozess-kommunikation
Übung
Kommunikation im Netzwerk
Interprozesskommunikation

AGENDA


Java
Nebenläufigkeit
Java Nebenläufigkeit
Java Interprozesskommunikation
Kommunikation im Netzwerk
Interprozesskommunikation
Verteilte Systeme

WAS SIE BIS HIERHIN GELERNT HABEN... SOLLTEN ☺

- Interessante Probleme erfordern parallele, verteilte Verarbeitung
  - Ein Rechner kann mehrere Verarbeitungen parallel erledigen
  - Mehrere Rechner können per Netzwerk Verarbeitungen verteilt erledigen
- Probleme, die bei paralleler und verteilter Verarbeitung auftreten
- Realisierung Nebenläufigkeit und Verteilung am Beispiel Java
- Technisch (!) können Sie alles entwickeln, was es an IT-Systemen gibt
  - Performante Datenbankserver
  - FileSharing Tauschbörsen
  - Enterprise-Resource-Planing(ERP)-Systeme
  - Hochverfügbare WebServer
  - Wissenschaftliche Simulationen (z.B. Strömungsverhalten)
- Aber es gibt mehr als nur eine technische Seite der Probleme!

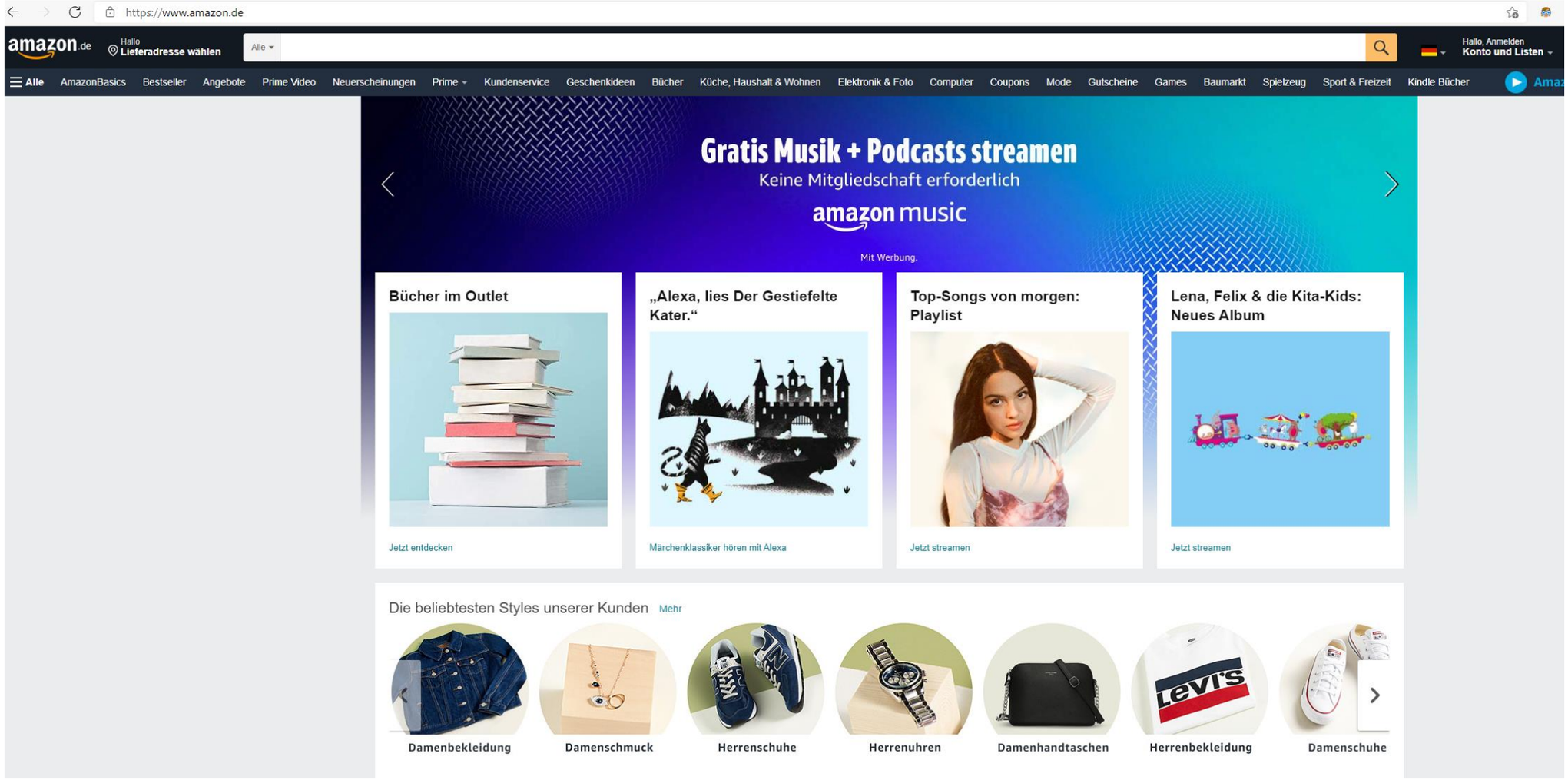
ZIELE

- Motivation und Herausforderungen
- Architektur
- Transparenz
- Gefahren verteilter Systeme
- Motivation Middleware

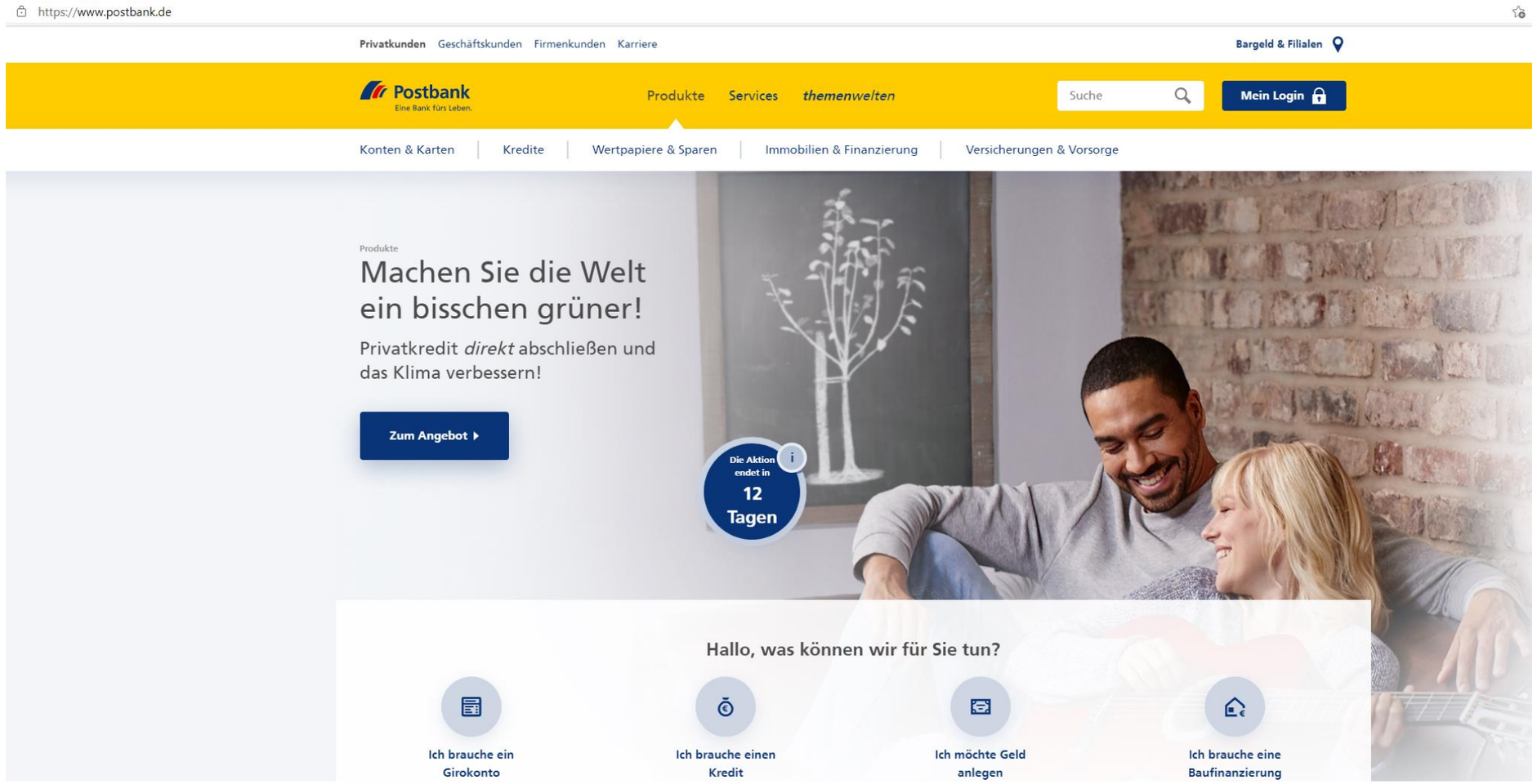
ÜBERSICHT

- Einführung
- Motivation und Herausforderungen
- Beispiel TicketShop
- Architektur
- Transparenz
- Middleware

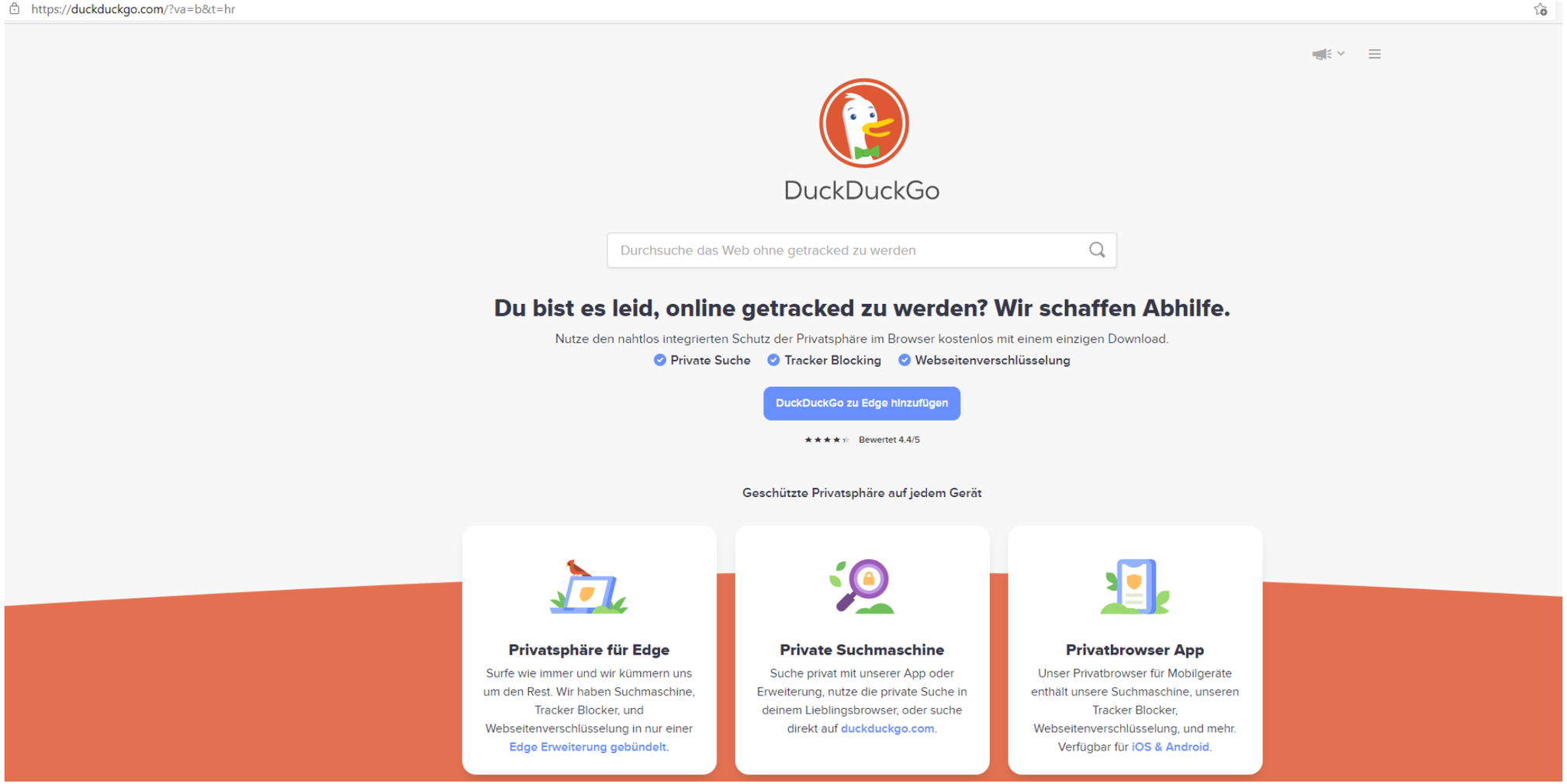
BEISPIEL E-COMMERCE



BEISPIEL E-COMMERCE



BEISPIEL INFORMATIONSSYSTEME

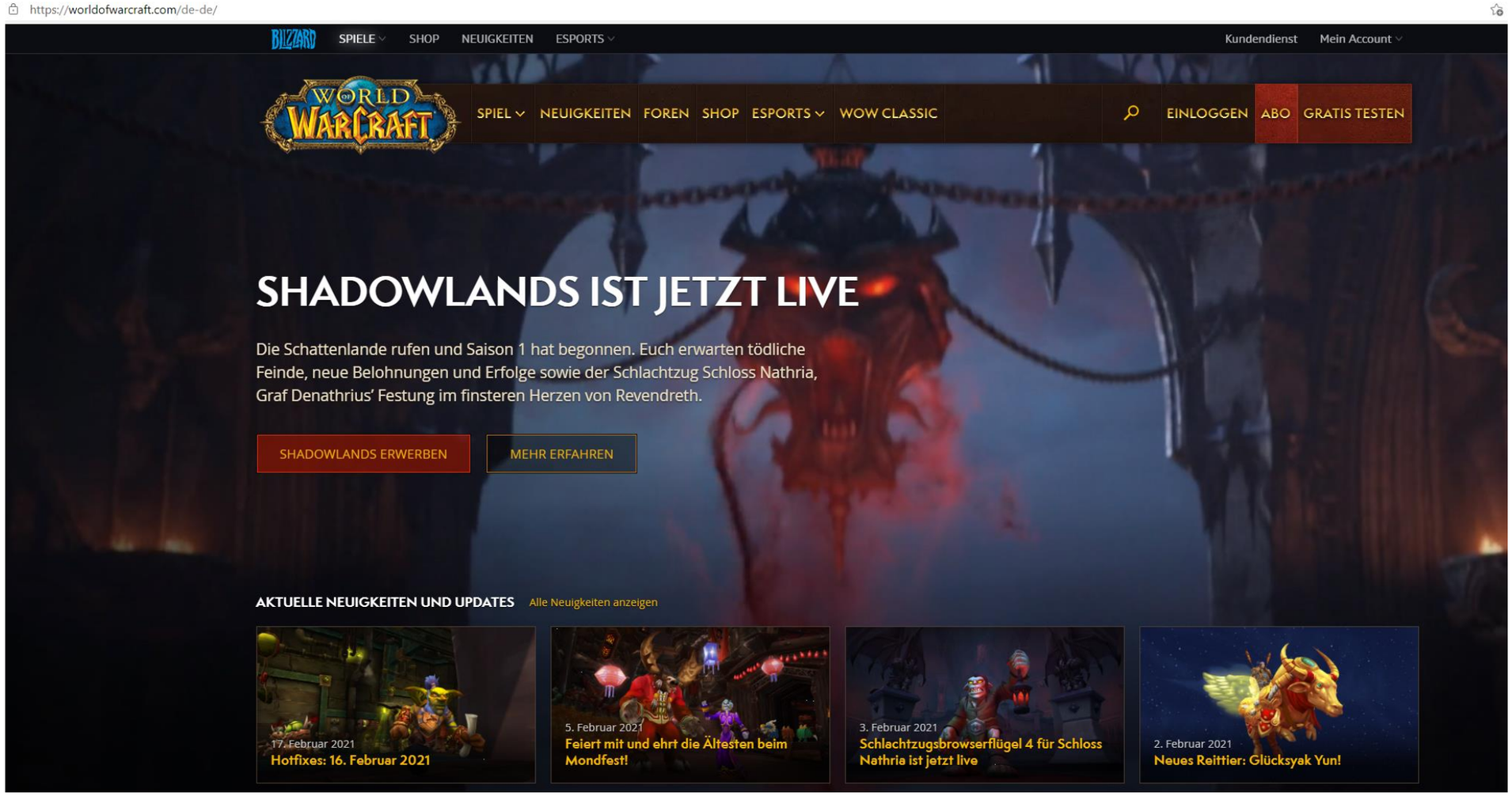


BEISPIEL GRID-COMPUTING (CERN)

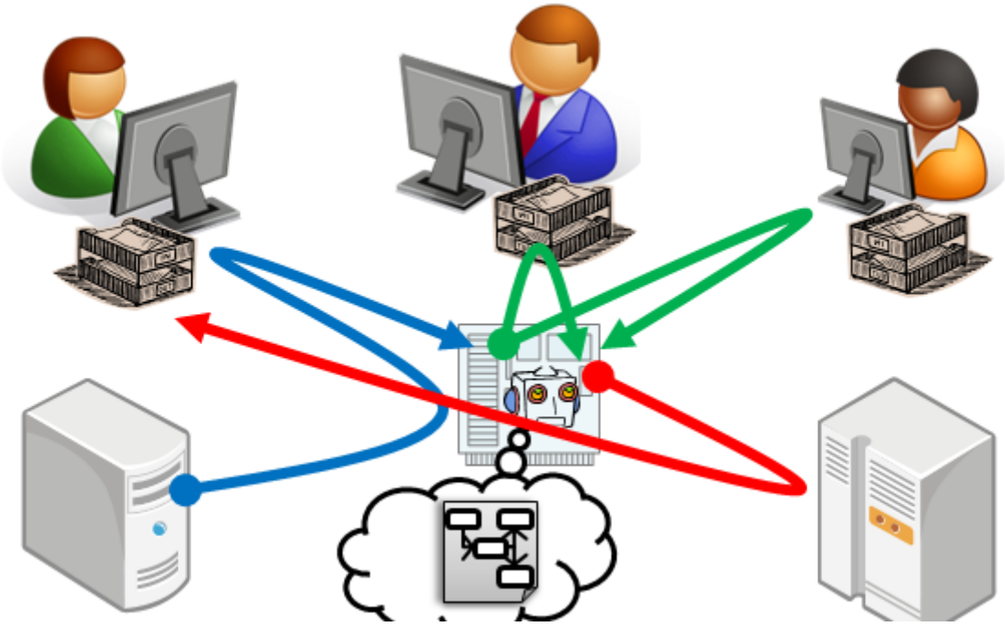
https://commons.wikimedia.org/wiki/File:CERN_Server_03.jpg

BEISPIEL SOZIALE NETZWERKE

BEISPIEL MULTIPLAYER-GAMES

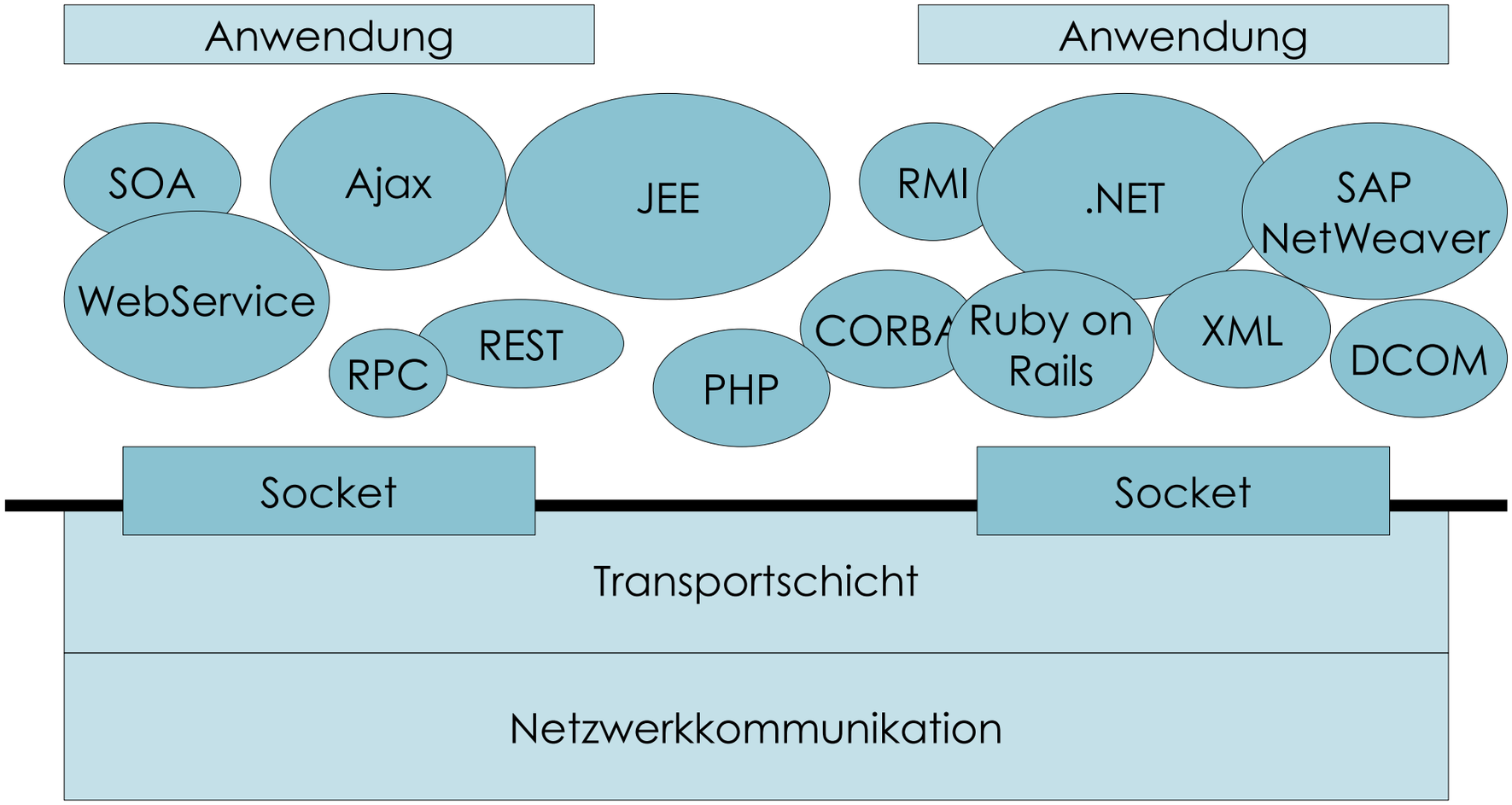


BEISPIEL GRUPPENARBEIT



BEISPIEL CLOUD COMPUTING

TECHNOLOGIEN



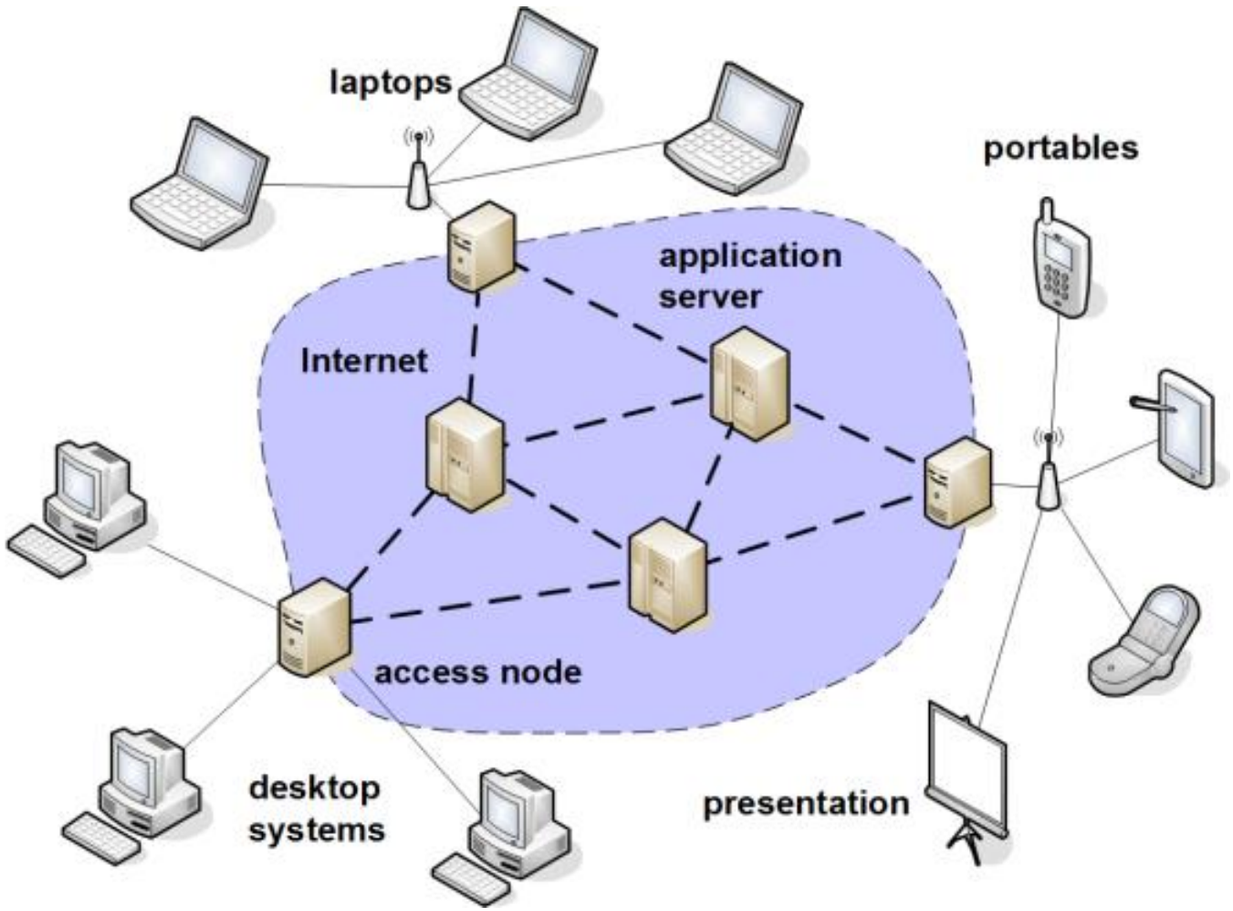
HISTORISCHE ENTWICKLUNG VON ANWENDUNGSSYSTEMEN

- Bis Mitte der 80er Jahre Dominanz zentralisierter Rechner
- Verdrängung durch leistungsfähigere vernetzte Arbeitsplatzrechner
- Entstehung von Rechnerverbundsystemen
- Aufgaben werden verteilt auf Hardware und Software
- Historisches Wachstum bedingt Heterogenität
  - Ausführungsplattform aus Betriebsystem, Tools, Frameworks
  - Redundante Funktionalität mit unterschiedlichen Leistungsparametern
  - Versionen, Versionen, Vers...
- Es entwickeln sich komplexe verteilte Anwendungssysteme
- Die Verbreitung des Internets fördert diesen Trend zusätzlich

DEFINITION VERTEILTES SYSTEM

- Zusammenschluss unabhängiger Rechner, die über ein Netzwerk miteinander kommunizieren
- Führen Anwendungen für die Benutzer transparent hinsichtlich der realen Verteilung und Verarbeitung im Rechnernetz aus
- Eigentliche Komplexität der Anwendung bleibt somit verborgen

VERTEILTES SYSTEM



http://www.uni-marburg.de/fb12/verteilte_systeme/forschung/pastproj/crossware

ÜBERSICHT

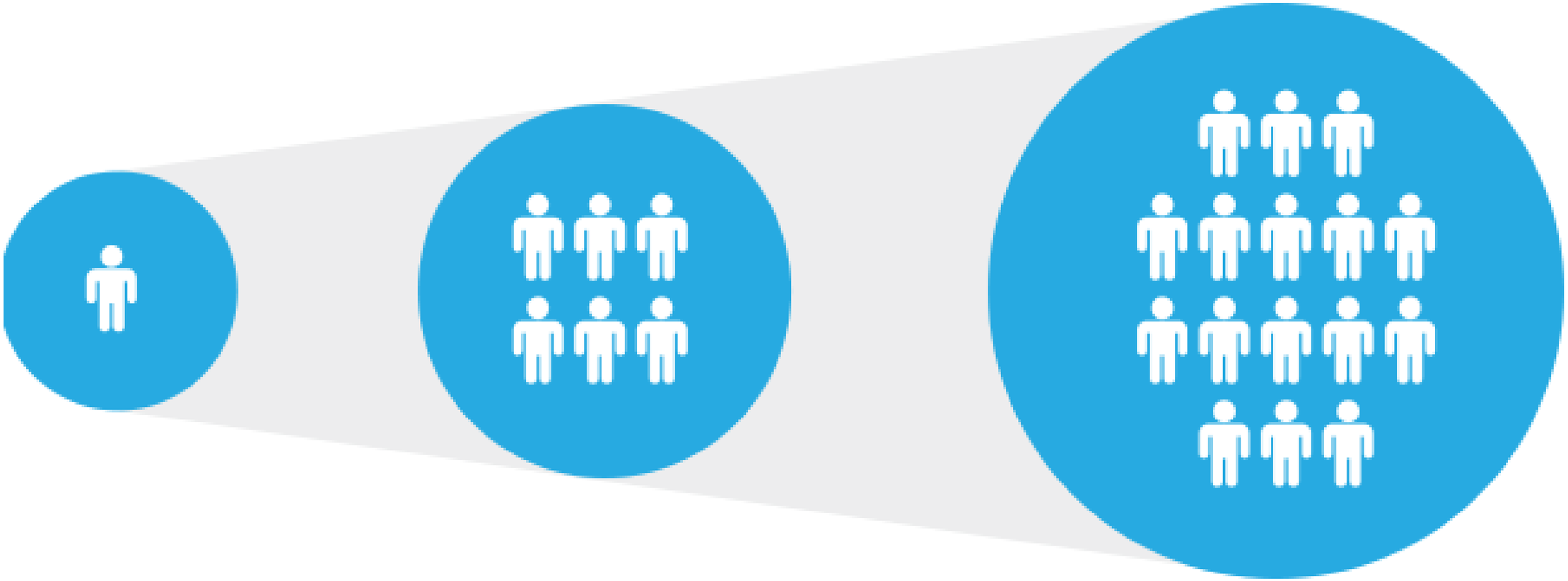
- Einführung
- Motivation und Herausforderungen
- Beispiel TicketShop
- Architektur
- Transparenz
- Middleware

MOTIVATION VERTEILTER SYSTEME

- Häufig ist nicht die Funktion des Systems Treiber für die Verteilung
- Sondern die *nicht-funktionalen* Anforderungen (NFA)
- Beispiel: Suchergebnis aus Datenbank zu Schlüsselwort finden

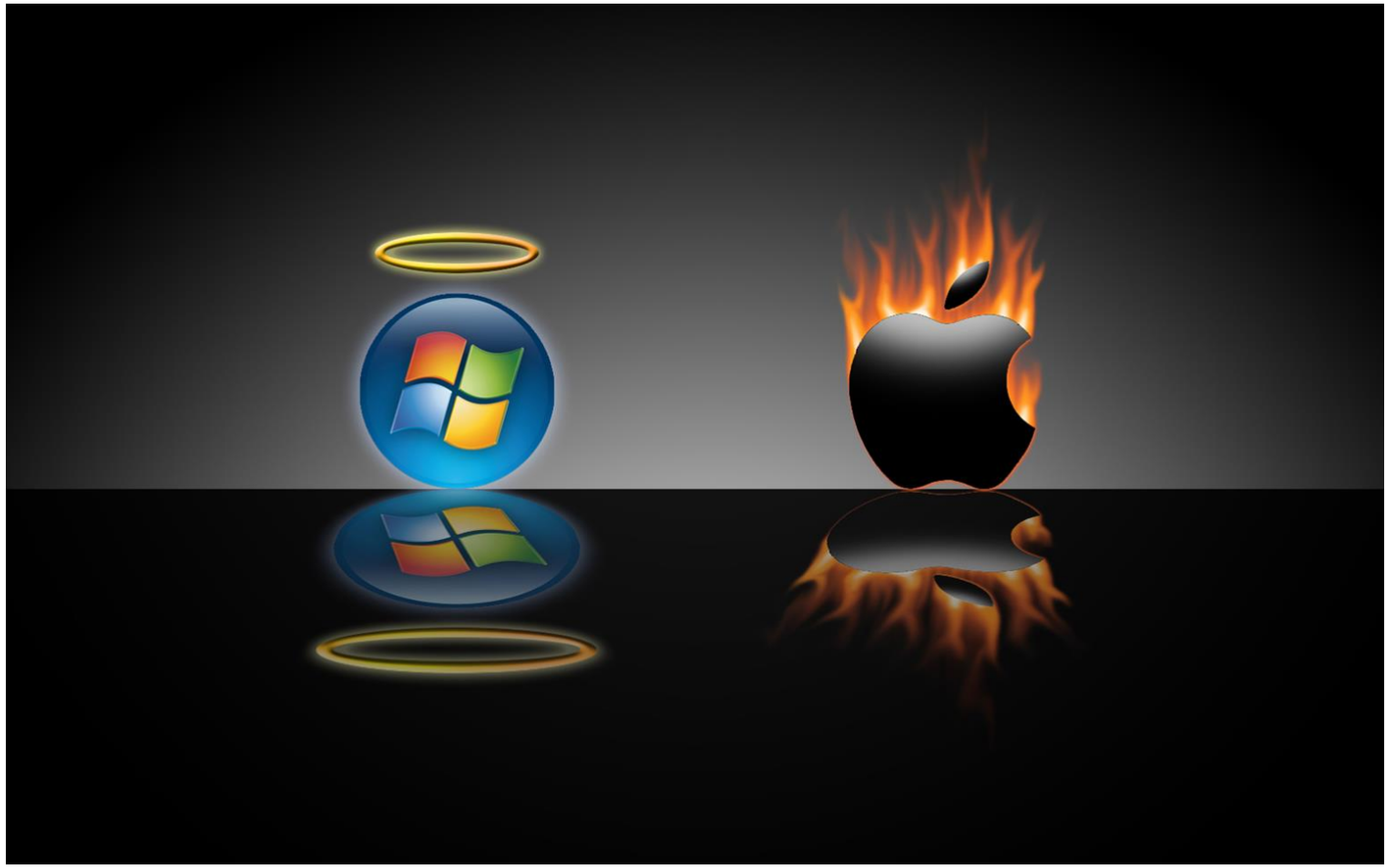
- Funktion Suchen/Finden der Einträge nach Schlüsselwort ist trivial
- Zusätzliche NFA:
  - Schneller als 1ms
  - Mehr 10000 User gleichzeitig
  - Lokalisation nach Sprache/Land
  - Ergebnis für diverse Ausgabemedien
  - Verfügbarkeit 24/7
  - schon interessanter
  - schon spannend
  - fordernd
  - aufwändig
  - umfangreich

SKALIERBARKEIT



PERFORMANZ

HETEROGENITÄT

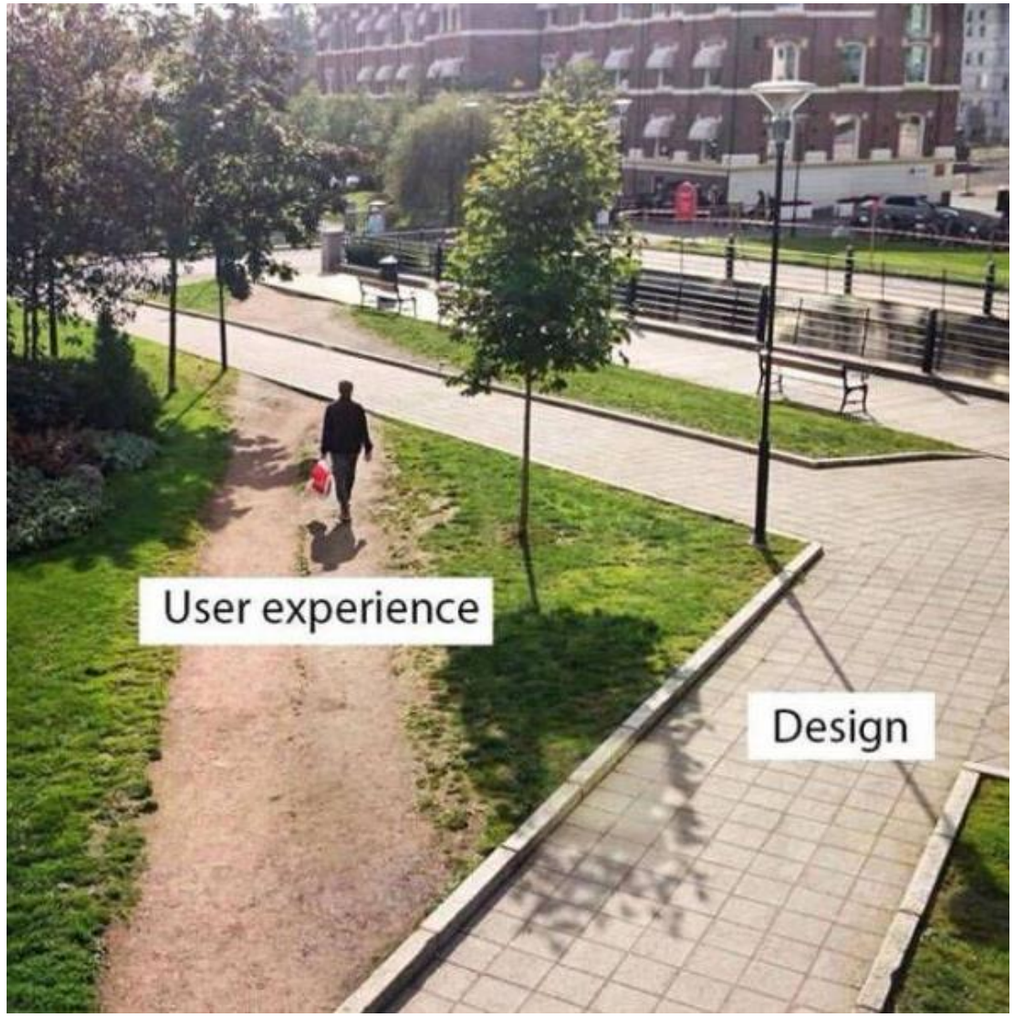


WARTBARKEIT UND ERWEITERBARKEIT

FEHLERTOLERANZ

USABILITY

USABILTY VS USEFULNESS/USER EXPERIENCE



ROBUSTHEIT

SICHERHEIT

HERAUSFORDERUNGEN AUS NFA

- NFA erfordern Verteilung auf mehrere Komponenten und Rechner
- Sinnvolle Verteilung der Aufgaben auf die Komponenten kritisch
  - Vorteile durch Verteilung realisieren
  - Redundanzen vermeiden
  - Minimale Abhängigkeiten, maximale Kopplung
- Sinnvolle Verteilung der Komponenten auf die Rechner kritisch
  - Keine/wenige Leistungsverluste
  - Vermeidung von Engstellen
- Beherrschen der Komplexität durch erhöhte Abhängigkeiten
- Erhöhte Sicherheitsrisiken gegenüber zentralisierten Lösungen

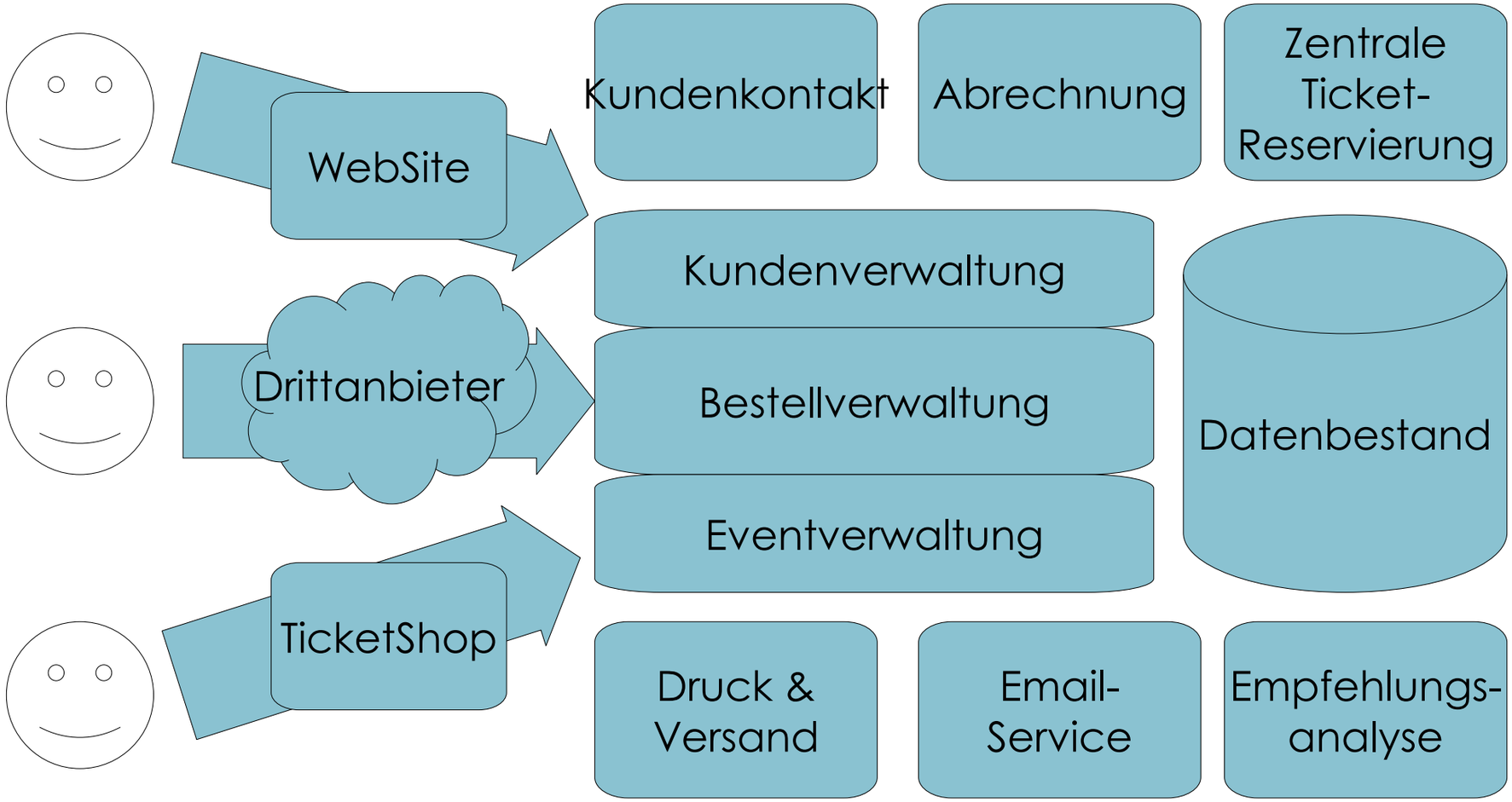
ÜBERSICHT

- Einführung
- Motivation und Herausforderungen
- Beispiel TicketShop
- Architektur
- Transparenz
- Middleware

ANWENDUNGSBEISPIEL TICKETSHOP

- Verkauf von Tickets für alle möglichen Events
  - Per WebSite direkt an Kunden
  - Per Drittanbieter indirekt an Kunden
  - Per Direktverkauf am Kartenschalter
- Besonderes Ereignis
  - „Neues Event“ mit hoher Nachfrage zu Zeitpunkt X
  - Bereitstellung zusätzlicher Ressourcen für Bearbeitung
  - Zu sonstigen Zeiten deutlich geringere Ressourcenauslastung
  - Ableitung nicht funktionaler Anforderung der Skalierbarkeit

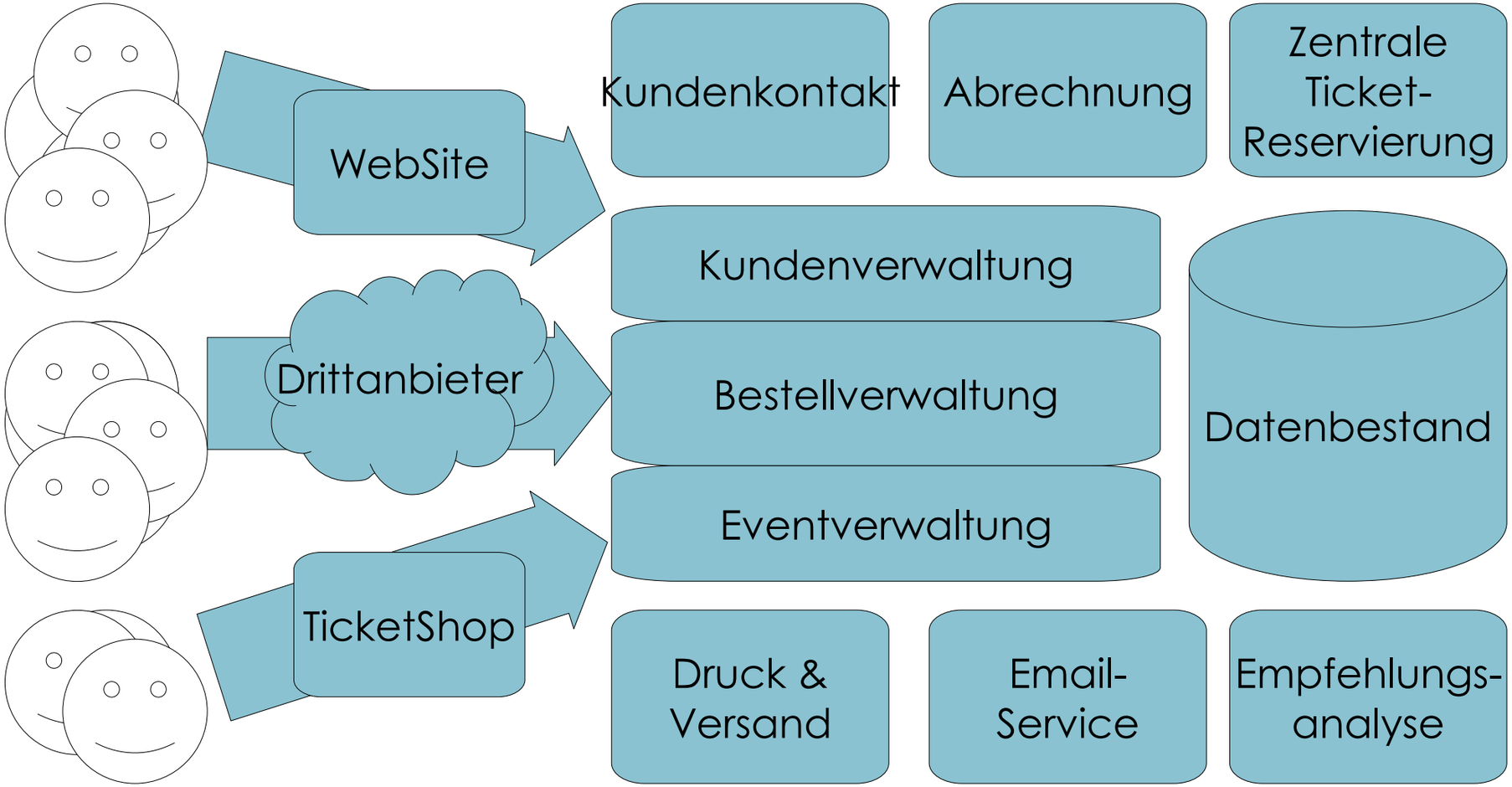
ÜBERSICHT FUNKTIONALE ANFORDERUNGEN



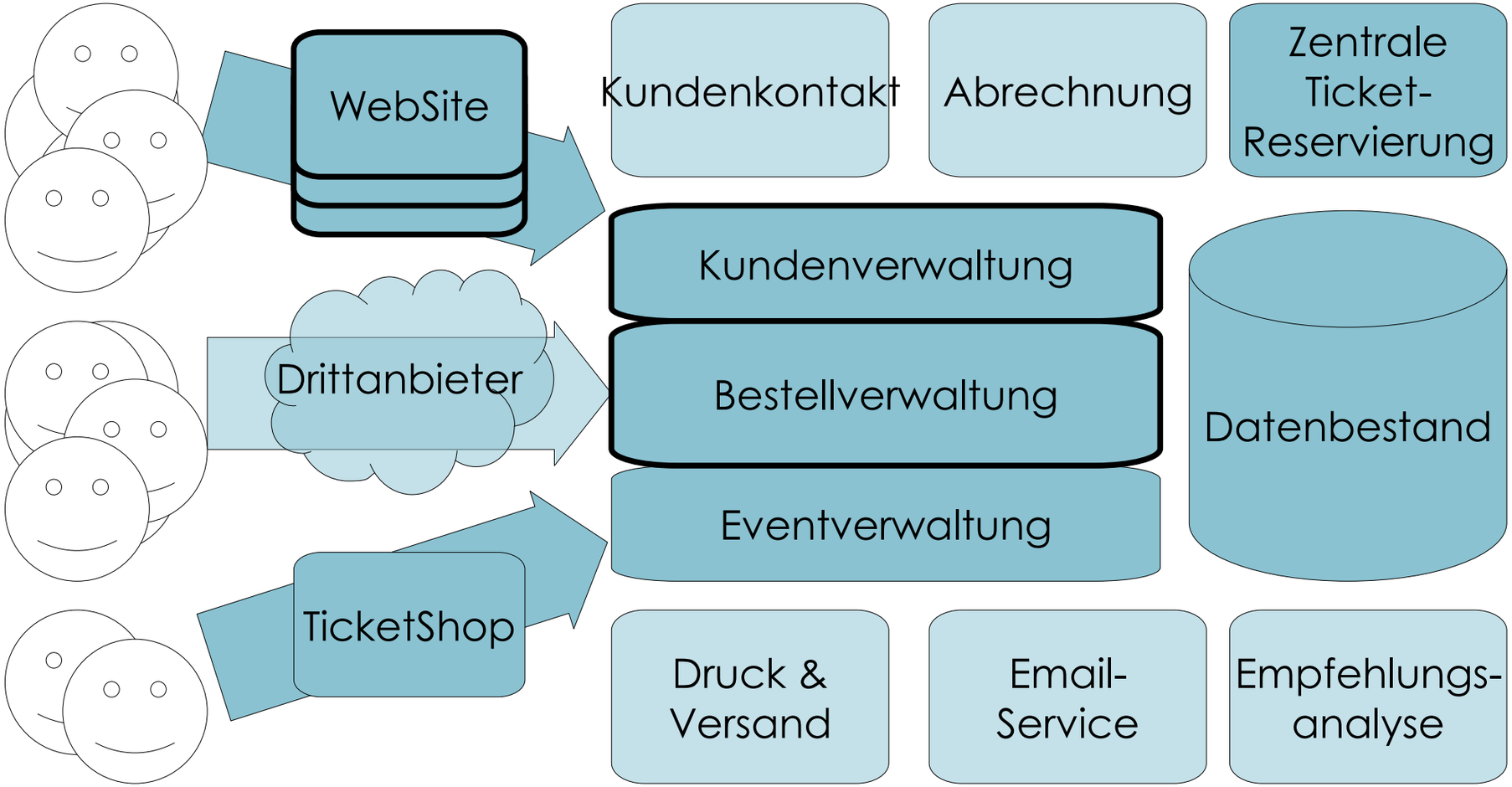
NICHT FUNKTIONALE ANFORDERUNGEN

- Skalierbarkeit
  - Zwischen 100 und 10000 Benutzern gleichzeitig
  - Direkter Kanal hat immer Vorrang
- Performanz
  - Bearbeitung der Anfragen per WebSite/Ticketshop in Echtzeit < 1ms
  - Drittanbieteranfragen <10s
- Heterogenität/Usability/Usefulness/Fehlertoleranz
  - Zugriff per Smartphone/Tablet/PC/Mac
- Verfügbarkeit
  - 7x24x365 - Keine Wartungsintervalle aus Kundensicht
- Sicherheit
  - Event- & Bestellverwaltung/Abrechnung getrennt vom Kunden

SONDERFALL NEUES EVENT



SONDERFALL NEUES EVENT



ÜBERSICHT

- Einführung
- Motivation und Herausforderungen
- Beispiel TicketShop
- Architektur
- Transparenz
- Middleware

ARCHITEKTUR VERTEILTER SYSTEME

- Verteilung von Funktionen und Realisierung von NFA erfordert einiges an Aufwand
- Planung und Konzeption der Verteilung im Vorfeld nötig
- Einteilung der Aufgaben in verschiedene Cluster
- Bauplan über die Verteilung der Cluster und Aufgaben
- Prinzip Teile und Herrsche und Trennung von Belangen
- Eine Architektur stellt die grundlegende Verteilung des Systems dar

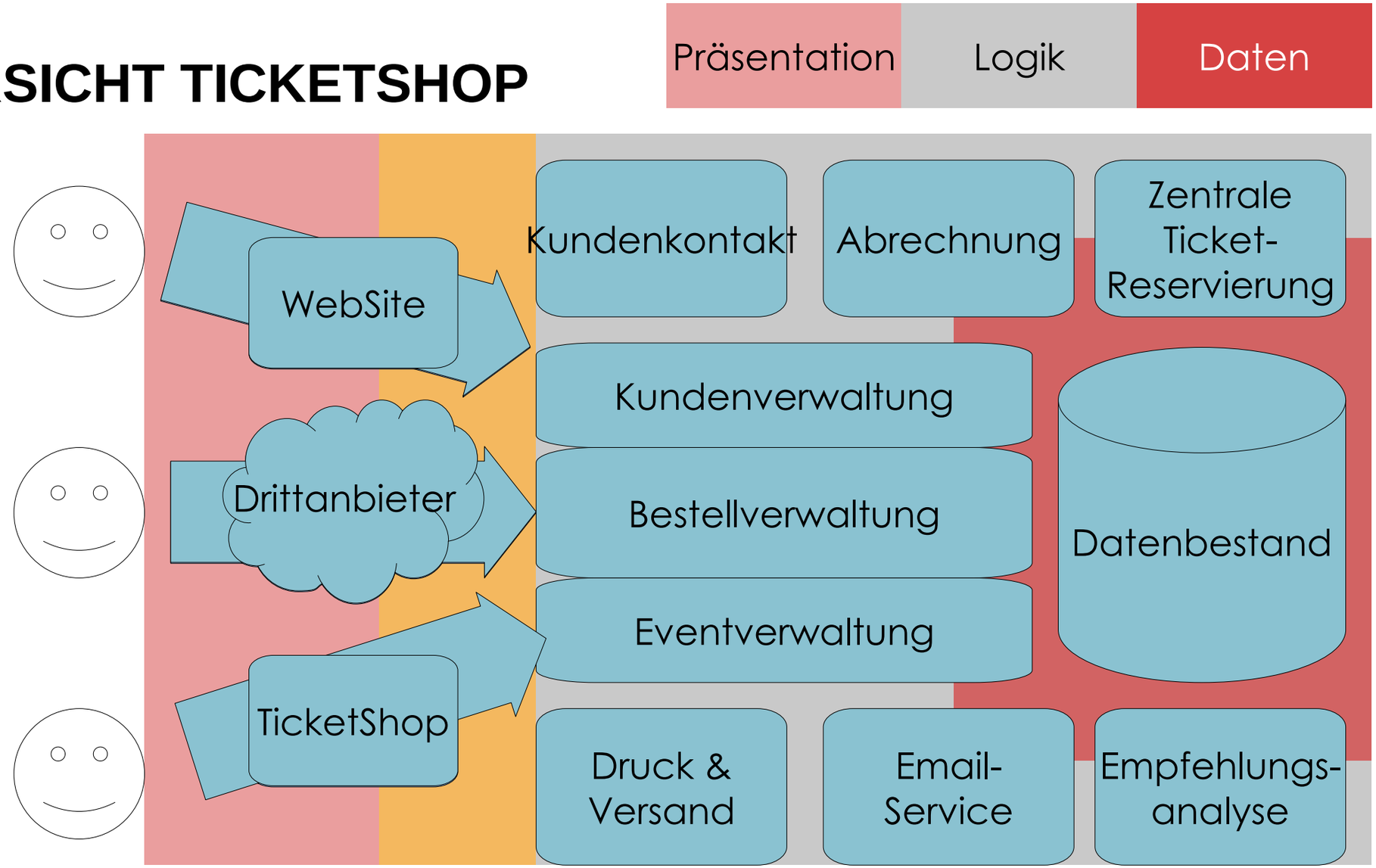
ANSATZ N-TIER ARCHITEKTUREN (*TIER* ENGL. EBENE)

- Grobe Aufteilung der Anwendung auf Komponenten
- Zuordnung der Komponenten auf Ebenen (*tier*)
- Ebenen bauen aufeinander auf und kommunizieren untereinander
- Ergänzung zum Client/Server-Modell
- Eine Ebene ist ein eigenständiger Prozessraum in der Anwendung
- In der Regel 2-, 3,- 4-Tier-Architekturen
- Trennung oft nicht explizit eindeutig erkennbar
- Oft eher ein konzeptioneller Ansatz zur Planung

ZENTRALE PROBLEMSTELLUNGEN IN VERTEILTEN INFORMATIONSSYSTEMEN

- Präsentation
  - Bereitstellung einer interaktiven Benutzerschnittstelle zur Anwendung
  - Abhängigkeit zu Ein- und Ausgabemedien
  - Aufbereitung der Ein- und Ausgabedaten für die Darstellung
  - Änderungsintensiv
- Anwendungslogik
  - Funktionalität einer Anwendung
  - Datenverarbeitung
  - Mittelfristige Änderungen
  - Idealerweise lediglich Kopplung zu Präsentation und Datenhaltung
- Datenhaltung
  - Persistente Speicherung der Daten und Sicherung der Datenkonsistenz
  - Langfristige Änderungen

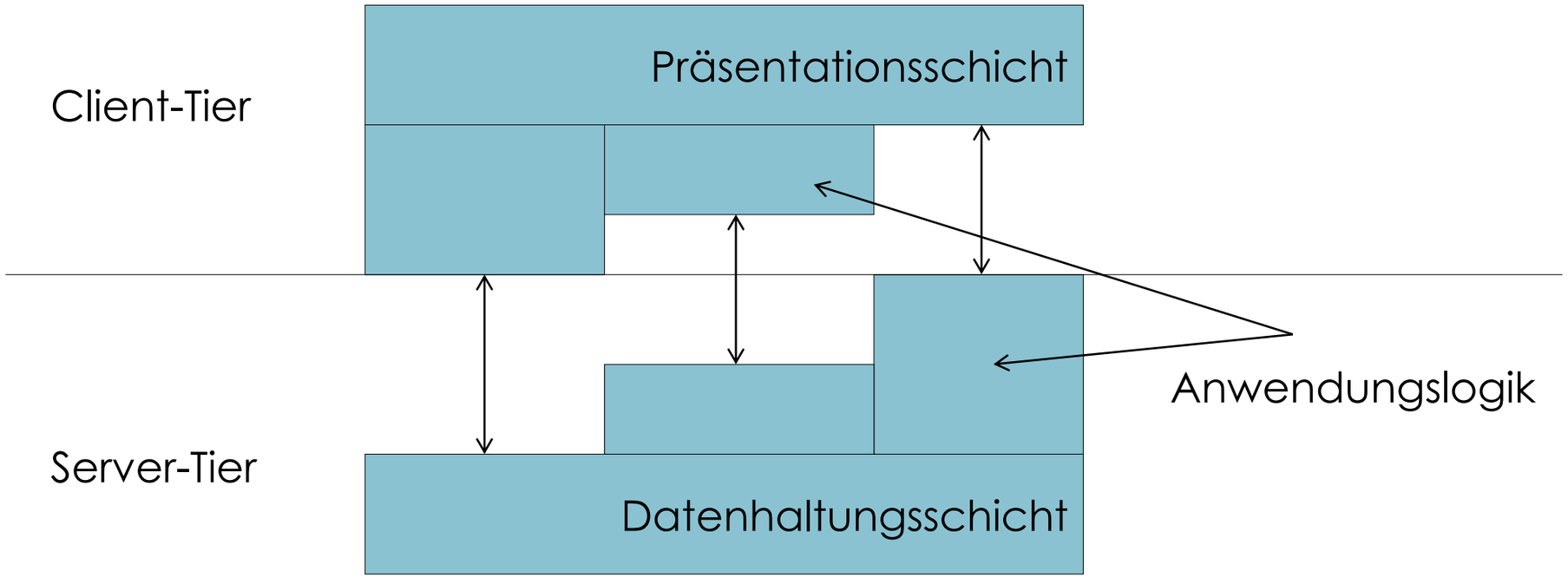
ÜBERSICHT TICKETSHOP



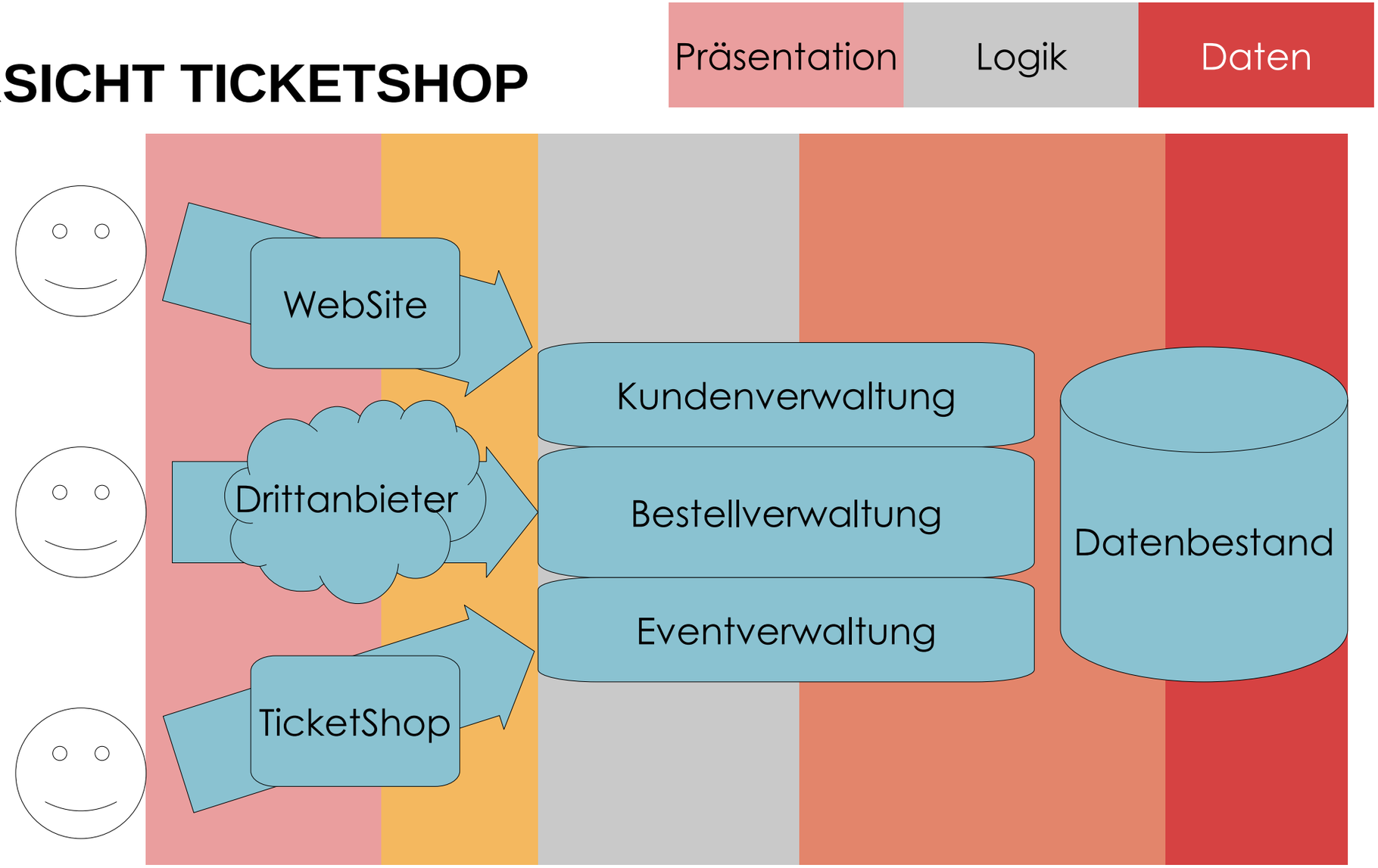
2-TIER-ARCHITEKTUR

- Einfaches Client/Server-Verteilungsmodell
- Client-Tier als Präsentationskomponente der Anwendung
- Server-Tier als Datenhaltungskomponente
- Keine eigene Ebene für Anwendungslogik
- Verteilung auf Client- und Server-Tier
- Vorteile
  - Komplexität für einfache Anwendungen beherrschbar
  - Aufwand bei einfachen Anwendungen überschaubar
- Nachteile
  - Schnell unübersichtlich → Schlecht wartbar, erweiterbar
  - Schlecht skalierbar → Schlimmstenfalls Reimplementierung

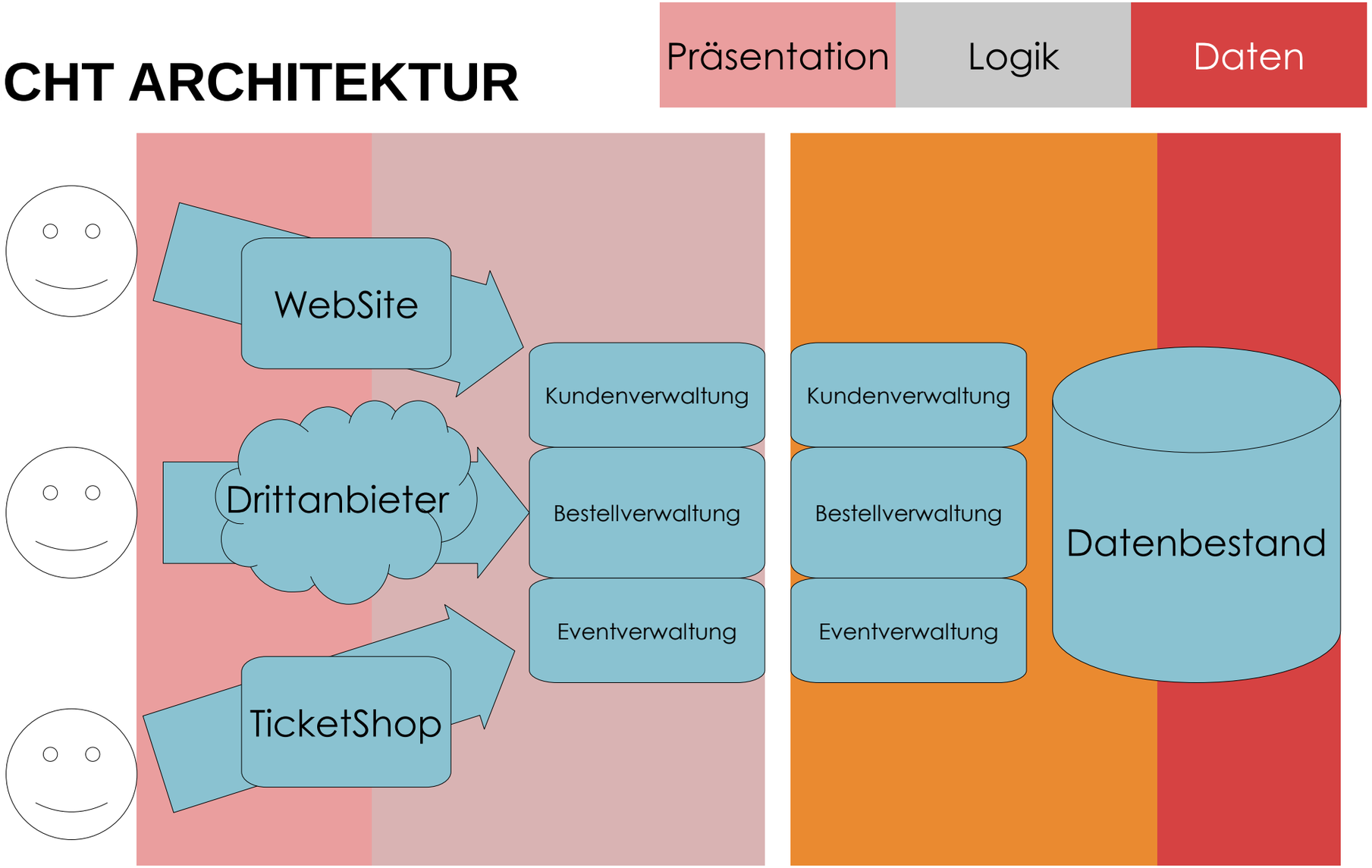
2-TIER-ARCHITEKTUR



ÜBERSICHT TICKETSHOP



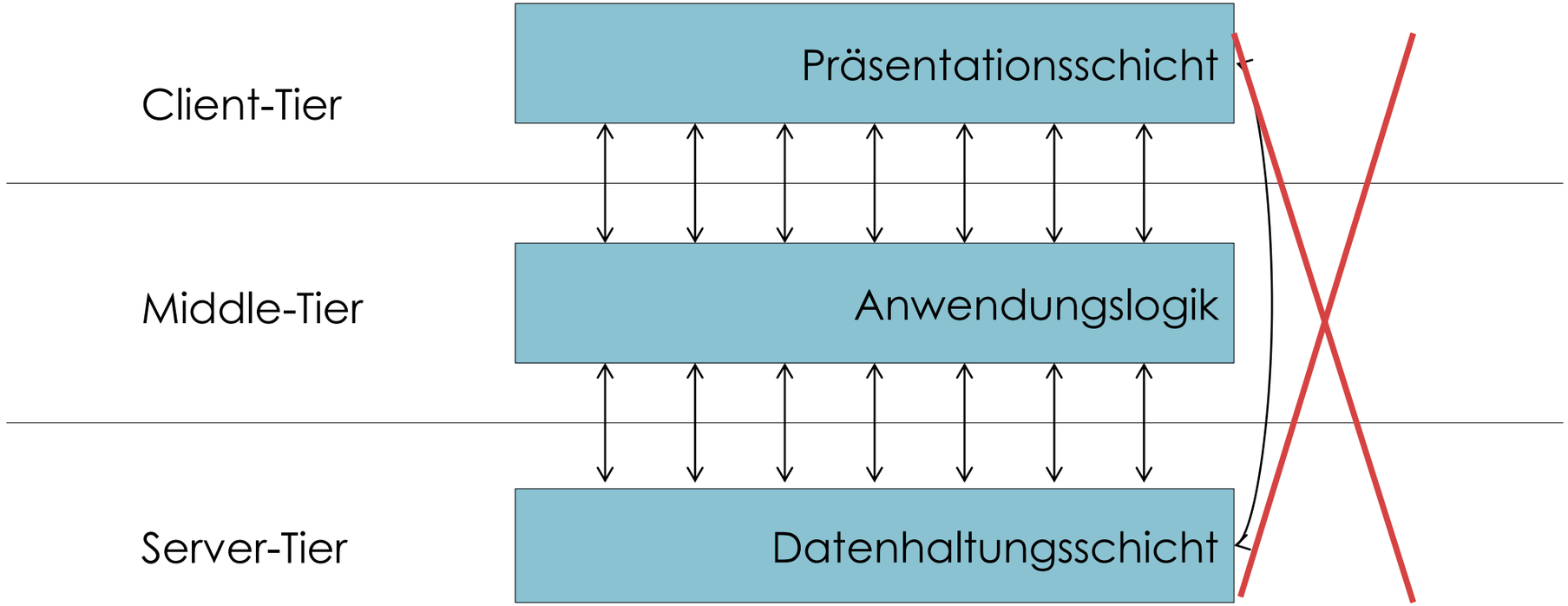
2 SCHICHT ARCHITEKTUR



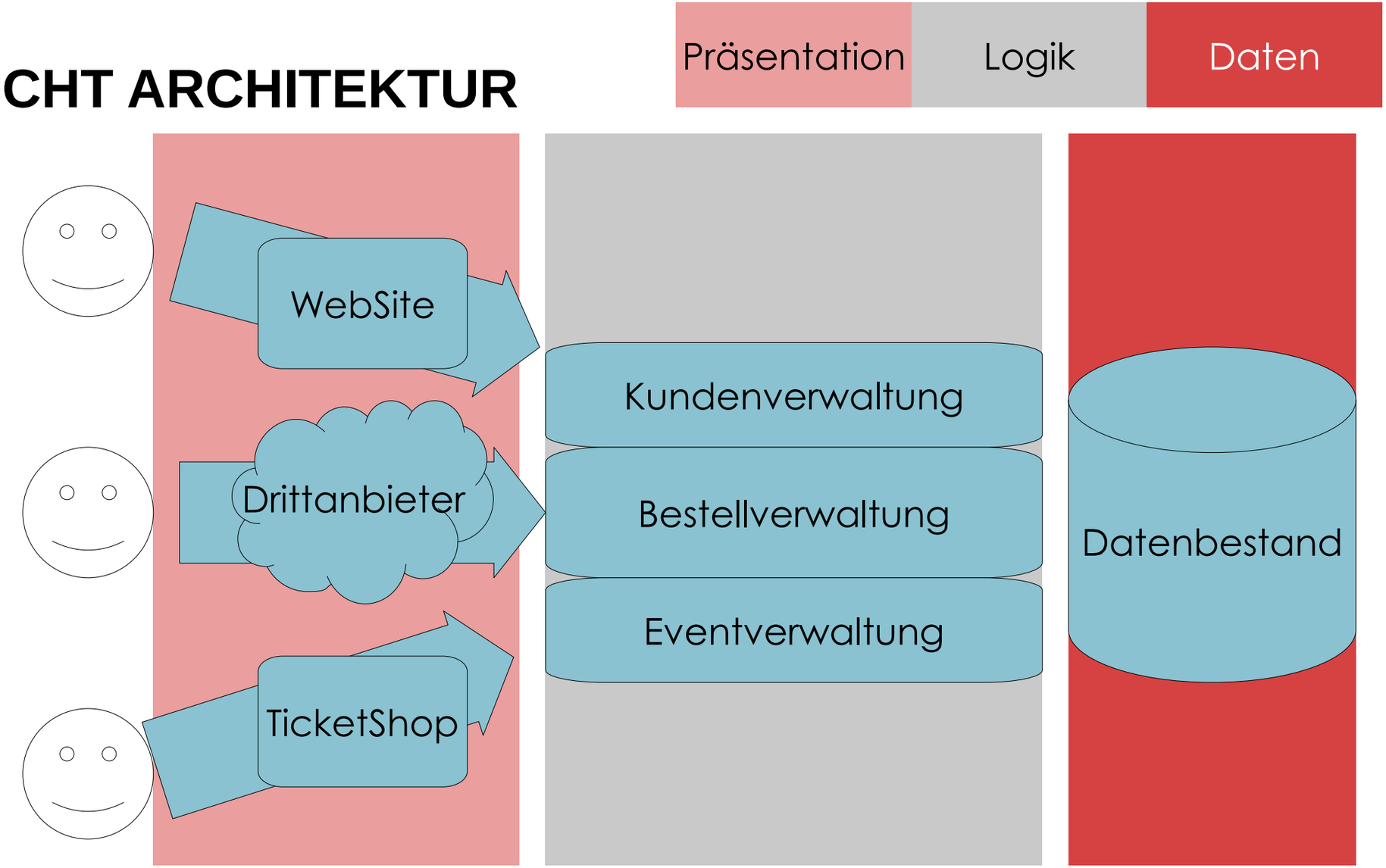
3-TIER-ARCHITEKTUR

- Erweiterung durch Middle-Tier für Anwendungslogik
- Beispiel: Web-Anwendung Content Managment System
  - Browser
  - Web-Server
  - Datenbank
- Vorteile
  - Zentrale Administration der Anwendungslogik
  - Gute Skalierbarkeit
- Nachteile
  - Overhead bei Kommunikation erfordert größeren Anwendungsumfang

3-TIER-ARCHITEKTUR



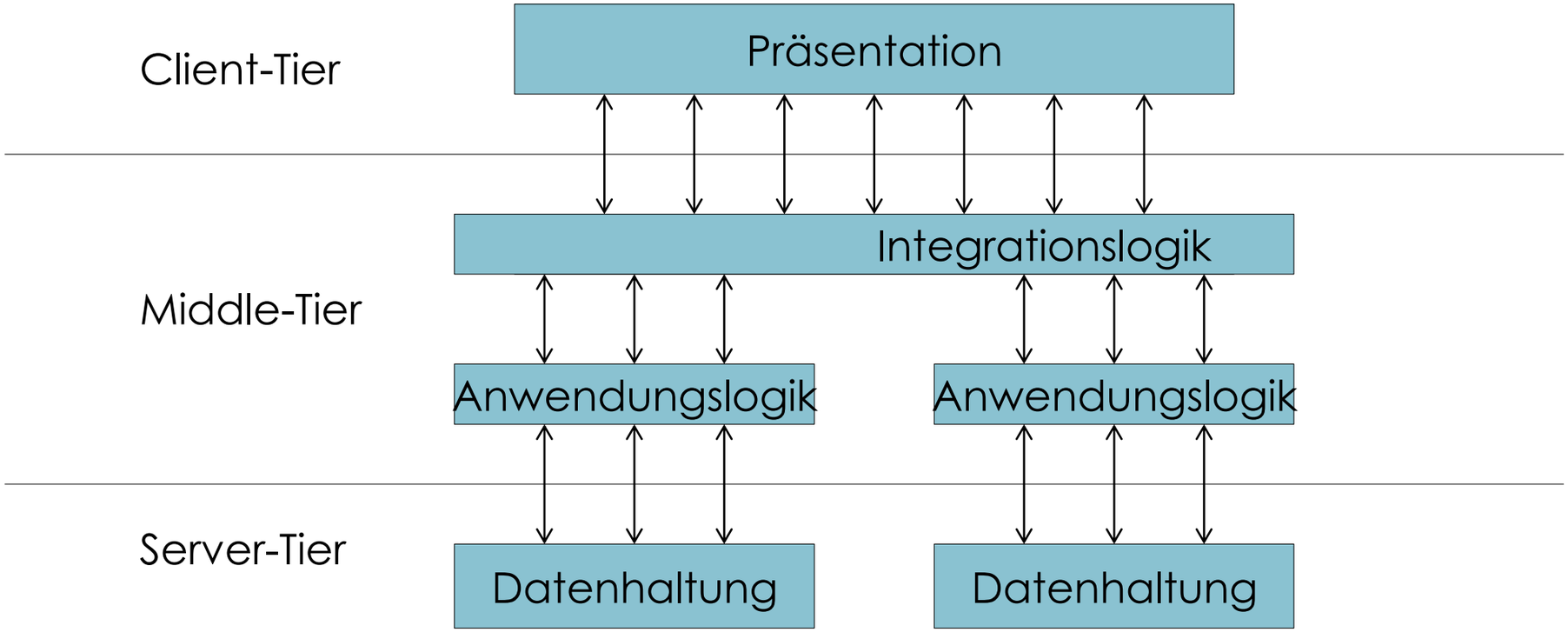
3 SCHICHT ARCHITEKTUR



4- UND N-TIER-ARCHITEKTUR

- Analog zu 3-Tier
- Weitere Verteilung der Anwendungslogik auf zusätzliche Schichten
- Beispiel Integrationsportal
  - Portal bietet einheitliche Oberfläche für mehrere heterogene Anwendungen der Organisation
  - Zusätzliche Schichten für Anwendungslogik und Datenhaltung der einzelnen Anwendungen
  - Eine Logik-Schicht realisiert Zuordnung/Konsolidierung der Anfragen zu den Anwendungen
  - Optional weitere Schicht zur Aufbereitung der Ausgabe auf verschiedene Endgeräte

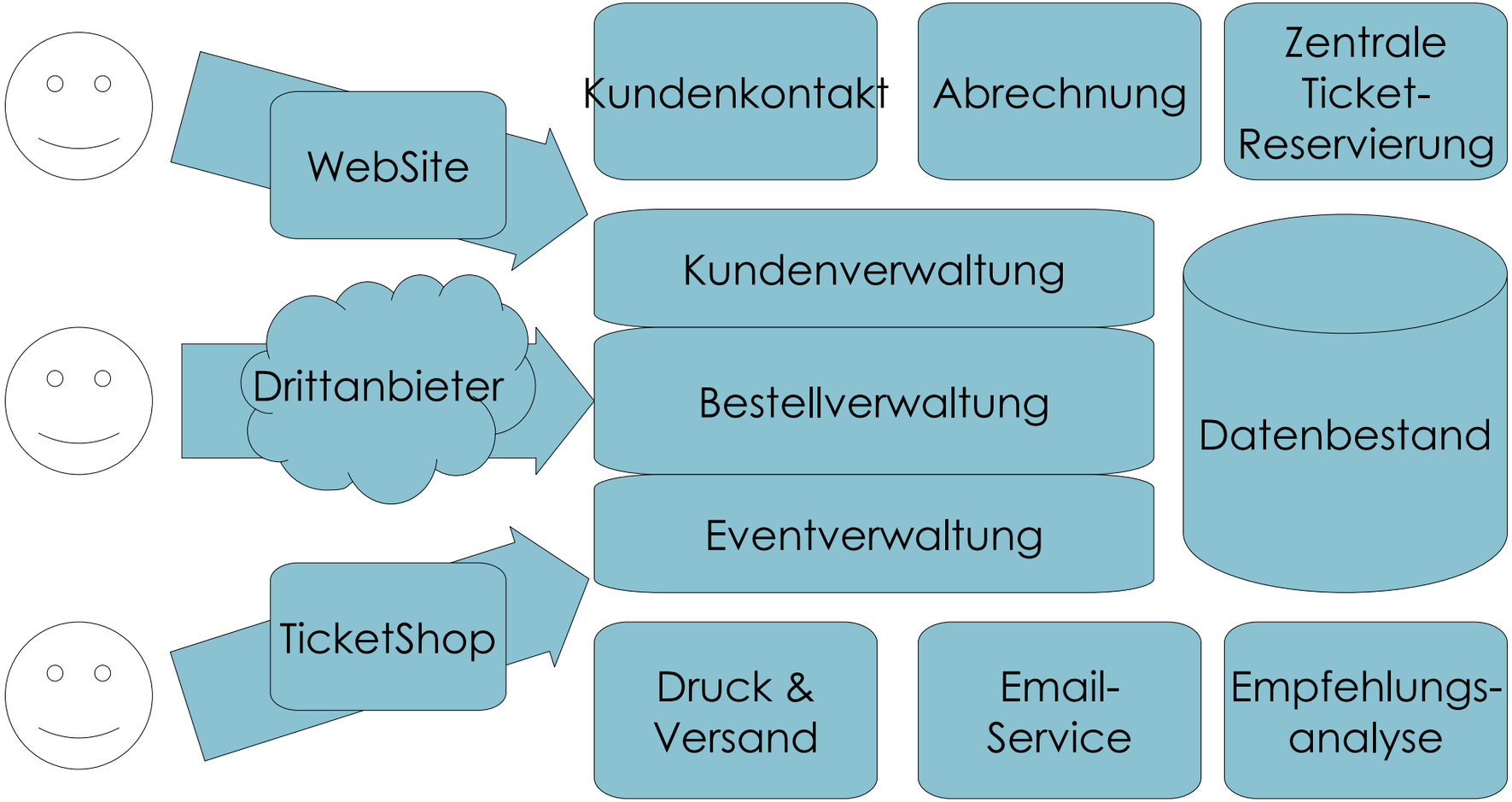
N-TIER-ARCHITEKTUR



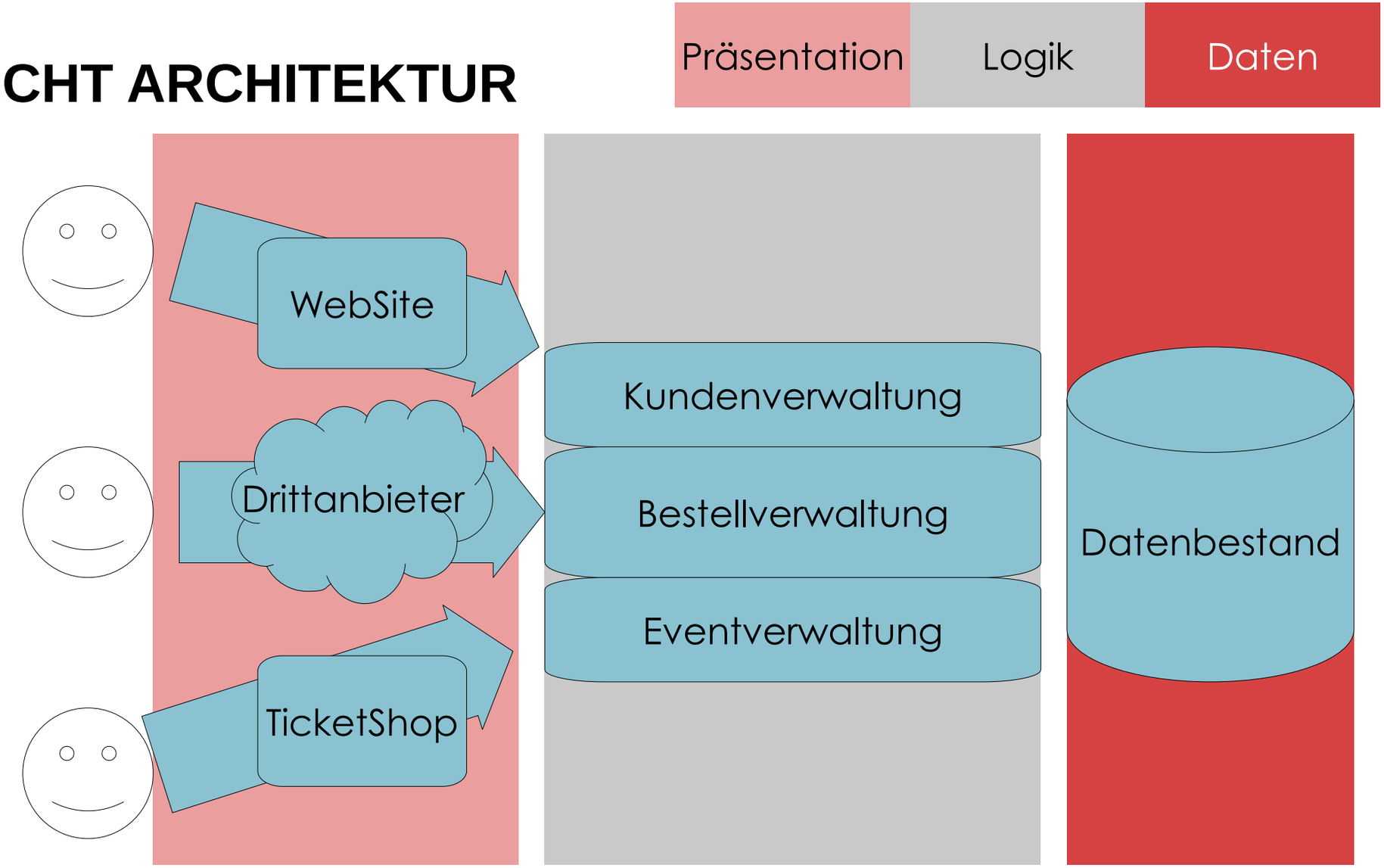
ÜBERSICHT

- Einführung
- Motivation und Herausforderungen
- Beispiel TicketShop
- Architektur
- Transparenz
- Middleware

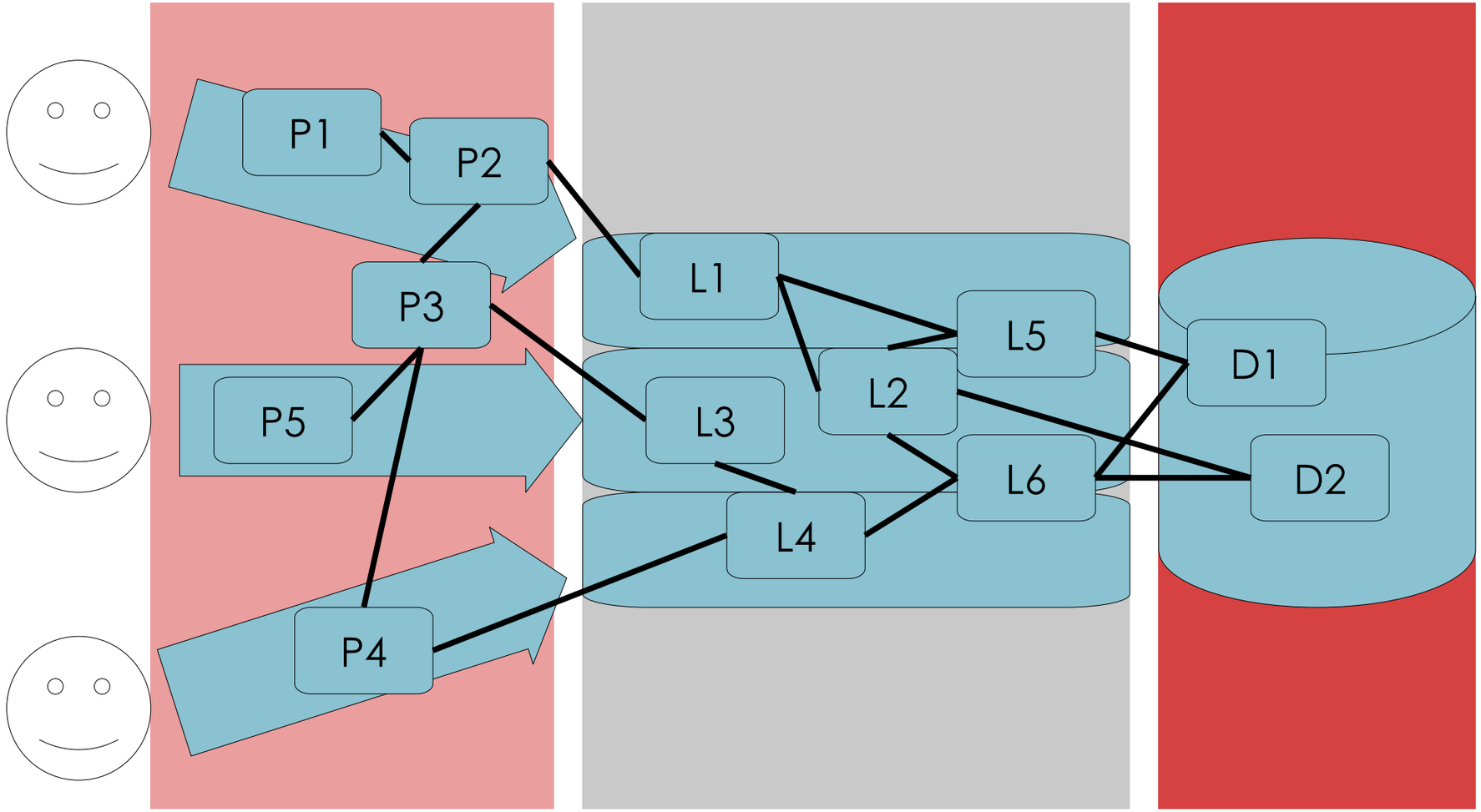
ÜBERSICHT TICKETSHOP



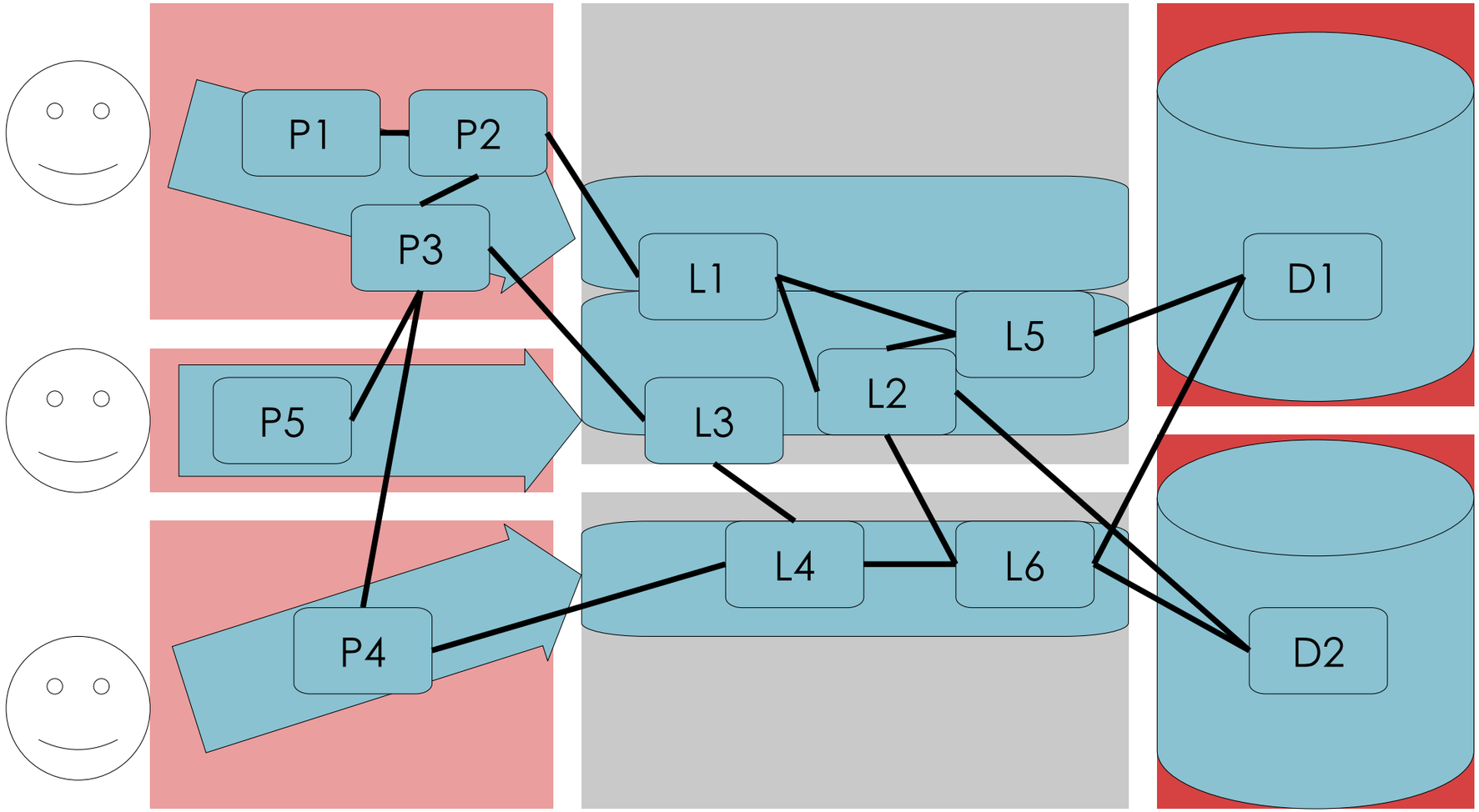
3 SCHICHT ARCHITEKTUR



VERTEILUNG AUF KOMPONENTEN



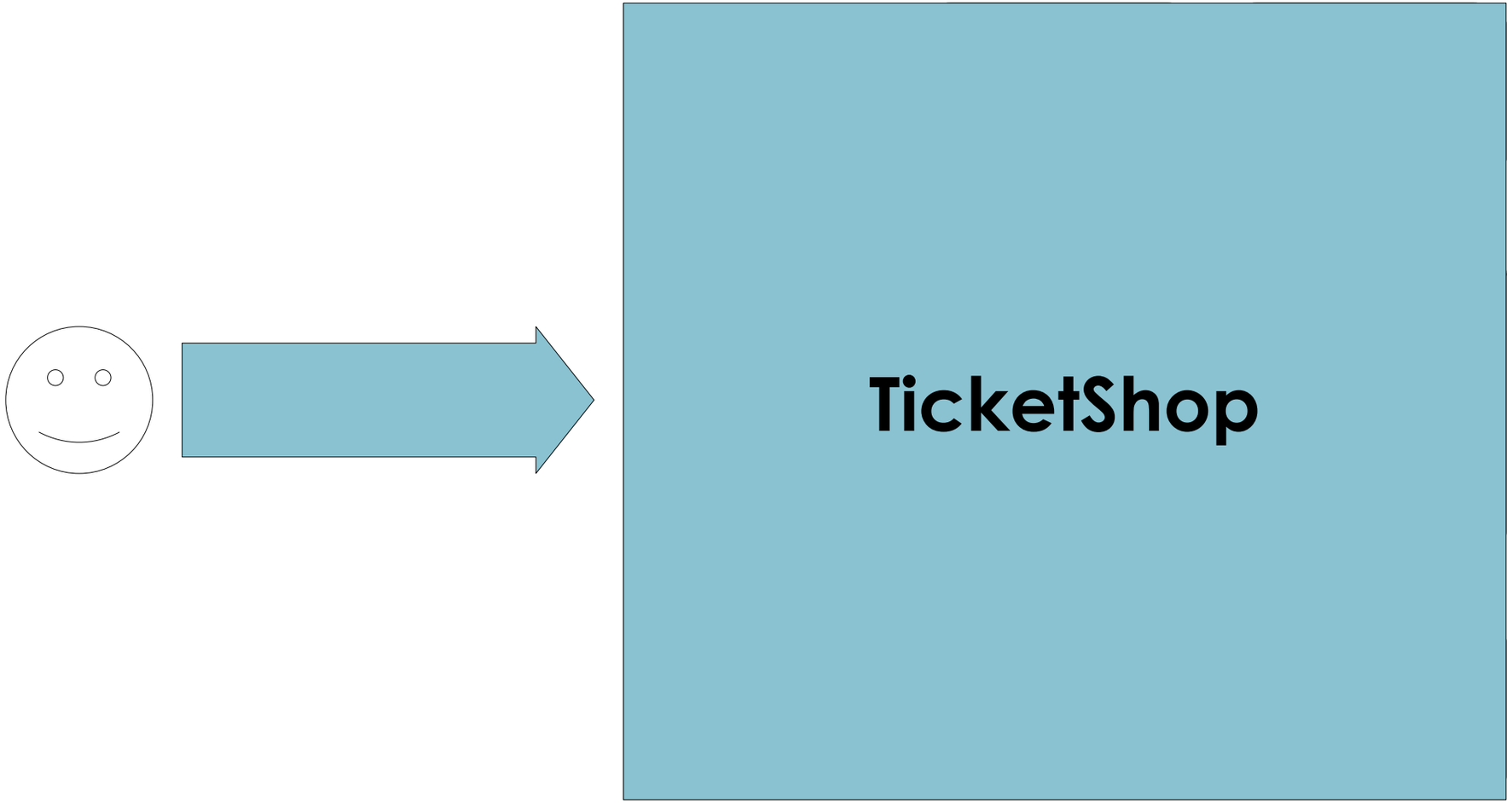
VERTEILUNG DER KOMPONENTEN AUF RECHNER



VERTEILUNG FÜR NUTZER TRANSPARENT

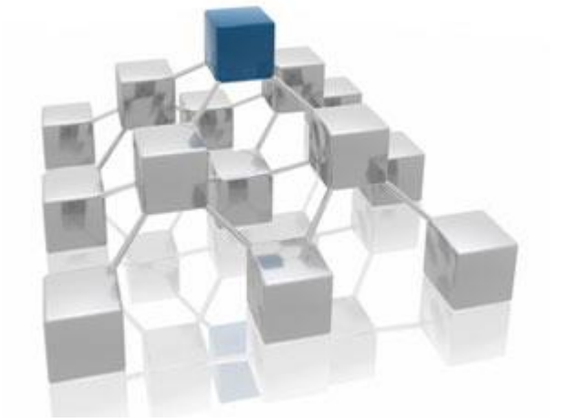
- Verteilung der Funktionalität für den einzelnen Nutzer uninteressant
- Die NFA sollen für ihn transparent realisiert werden
- Transparenz als „vollständige Transparenz“ und somit nicht sichtbar
- System aus verteilten Komponenten erscheint dem Nutzer als Einheit
- Auch aus Entwicklersicht Verteilungstransparenz interessant
- Realisierung durch standardisiertes und robustes Framework
- Entwickler kann sich auf Realisierung der Funktionalität konzentrieren

TRANSPARENTES VERTEILTES SYSTEM

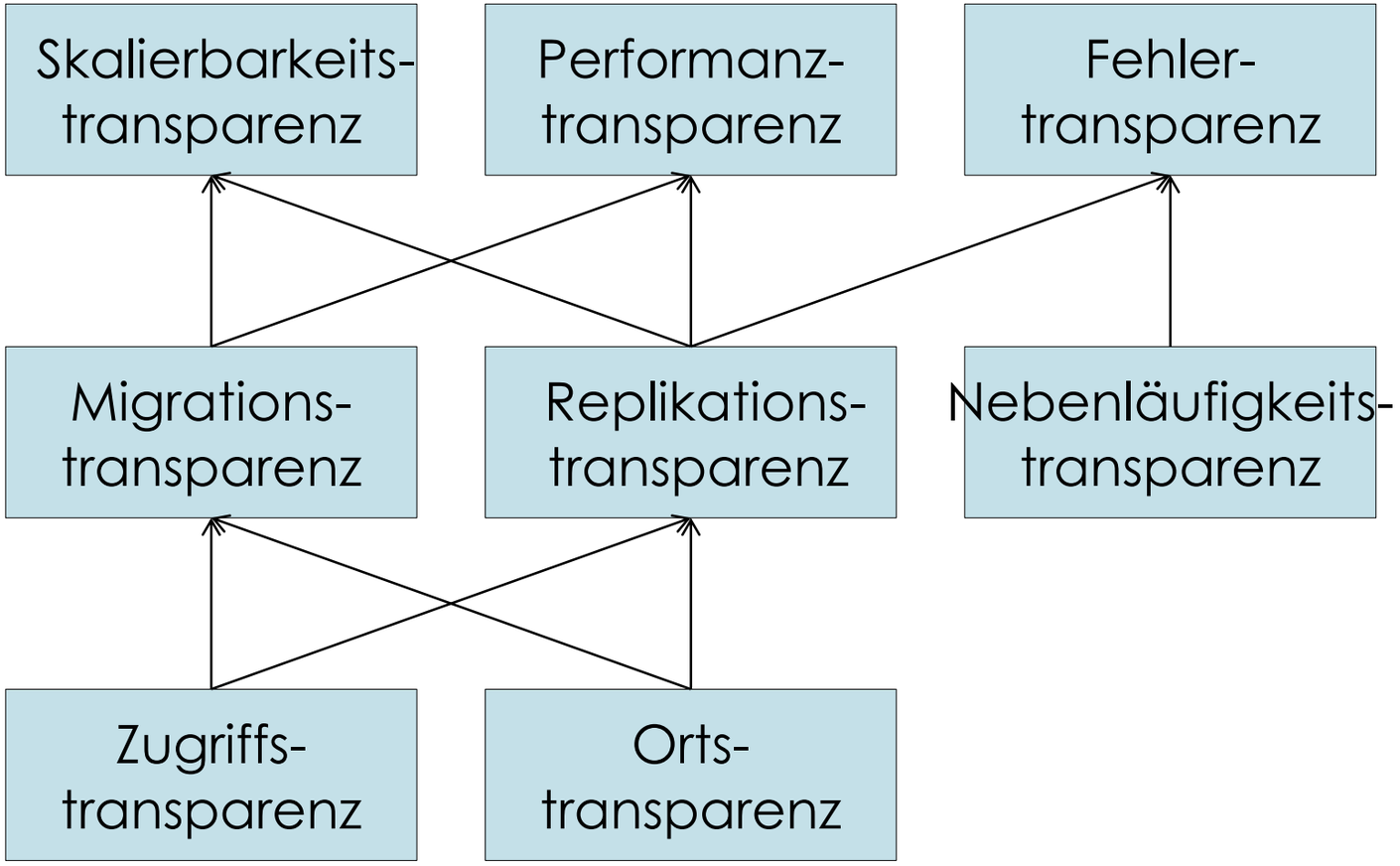


VERTEILUNGSTRANSPARENZ

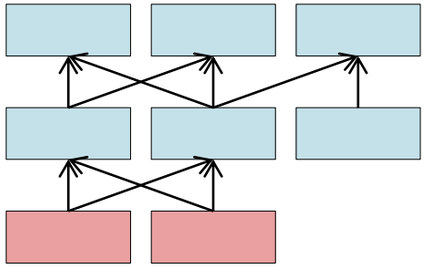
- Übergeordnete allgemeine Transparenzform
- „Abschirmung“ vor Komplexität und den technischen Details
- Betrifft Benutzer, Entwickler und Anwendungskomponenten selbst
- Gewährleistung einer homogenen Sicht auf das System
- Unterteilt in voneinander abhängige Tranzparenzdimensionen



DIMENSIONEN DER TRANSPARENZ



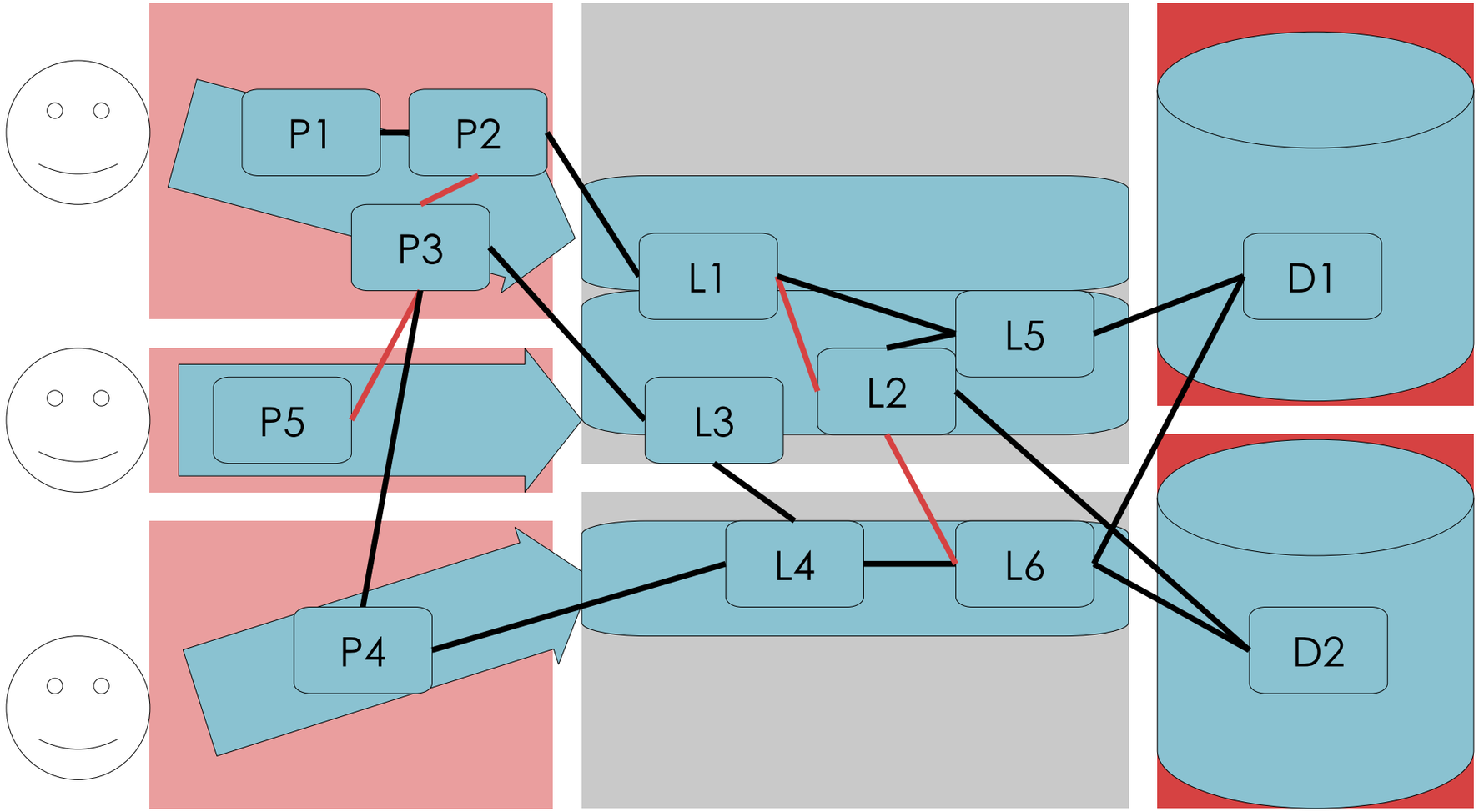
Quelle: Emmerich, W.: Konstruktion von verteilten Objekten, S. 27

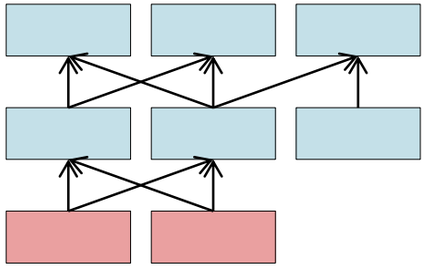


ZUGRIFFSTRANSPARENZ

- Zugriff auf lokale und entfernte Komponenten erfolgt auf gleiche Art
- Abstrahieren von Art der Datenrepräsentation
- Abstrahieren von Aufrufmechanismus Prozeduren/Funktionen/Methoden

ZUGRIFFSTRANSPARENZ

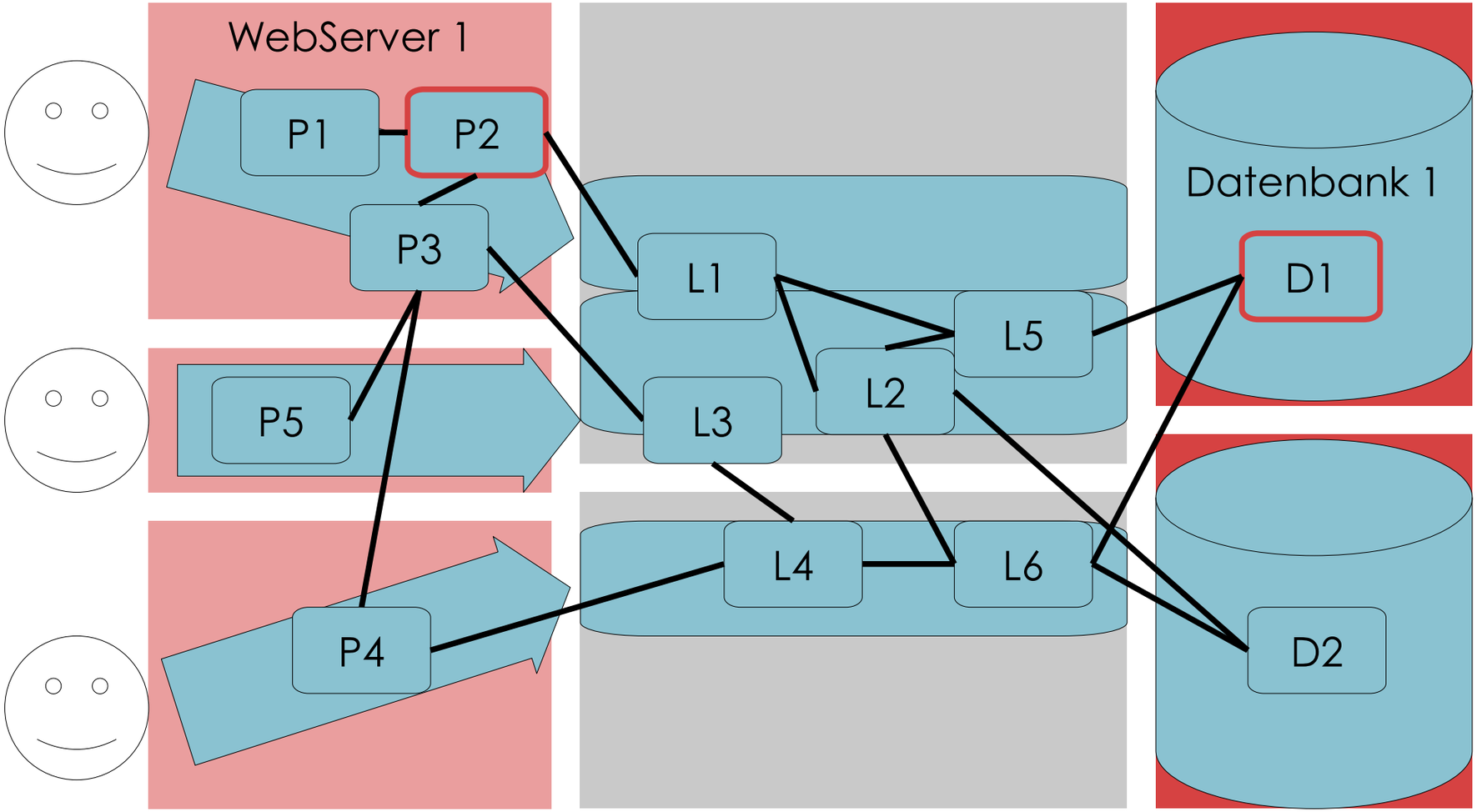




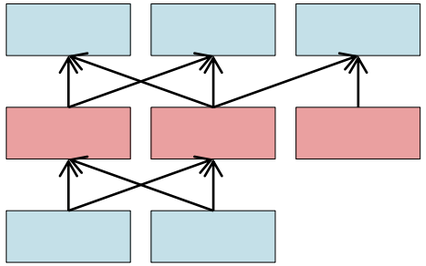
ORTSTRANSPARENZ

- Zugriff erfolgt über global eindeutige Namen ohne technische Details
- Der physische Ort von Komponenten ist dem Aufrufer nicht bekannt
- Lokalisierung wird vom verteilten System übernommen

ORTSTRANSPARENZ

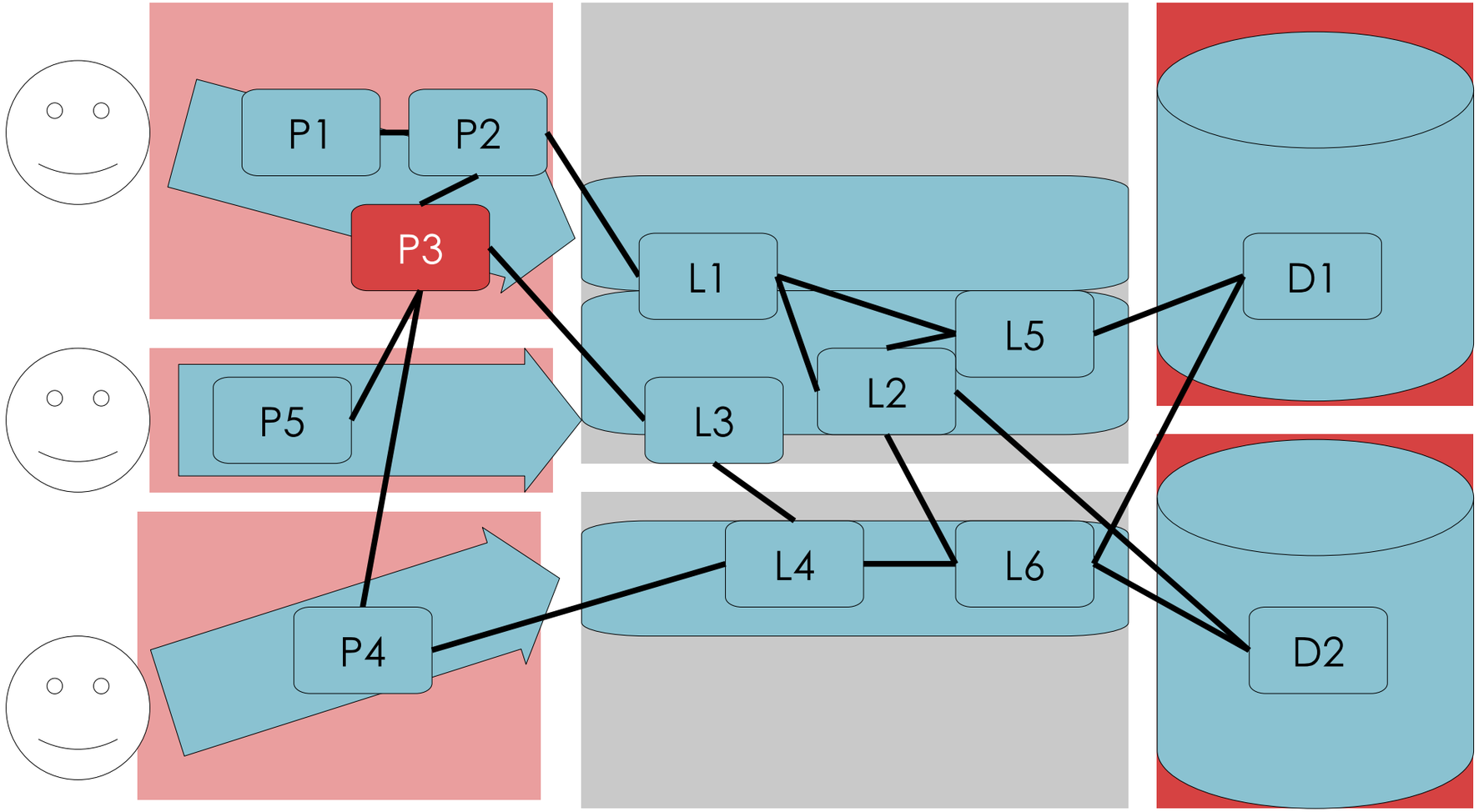


MIGRATIONSTRANSPARENZ

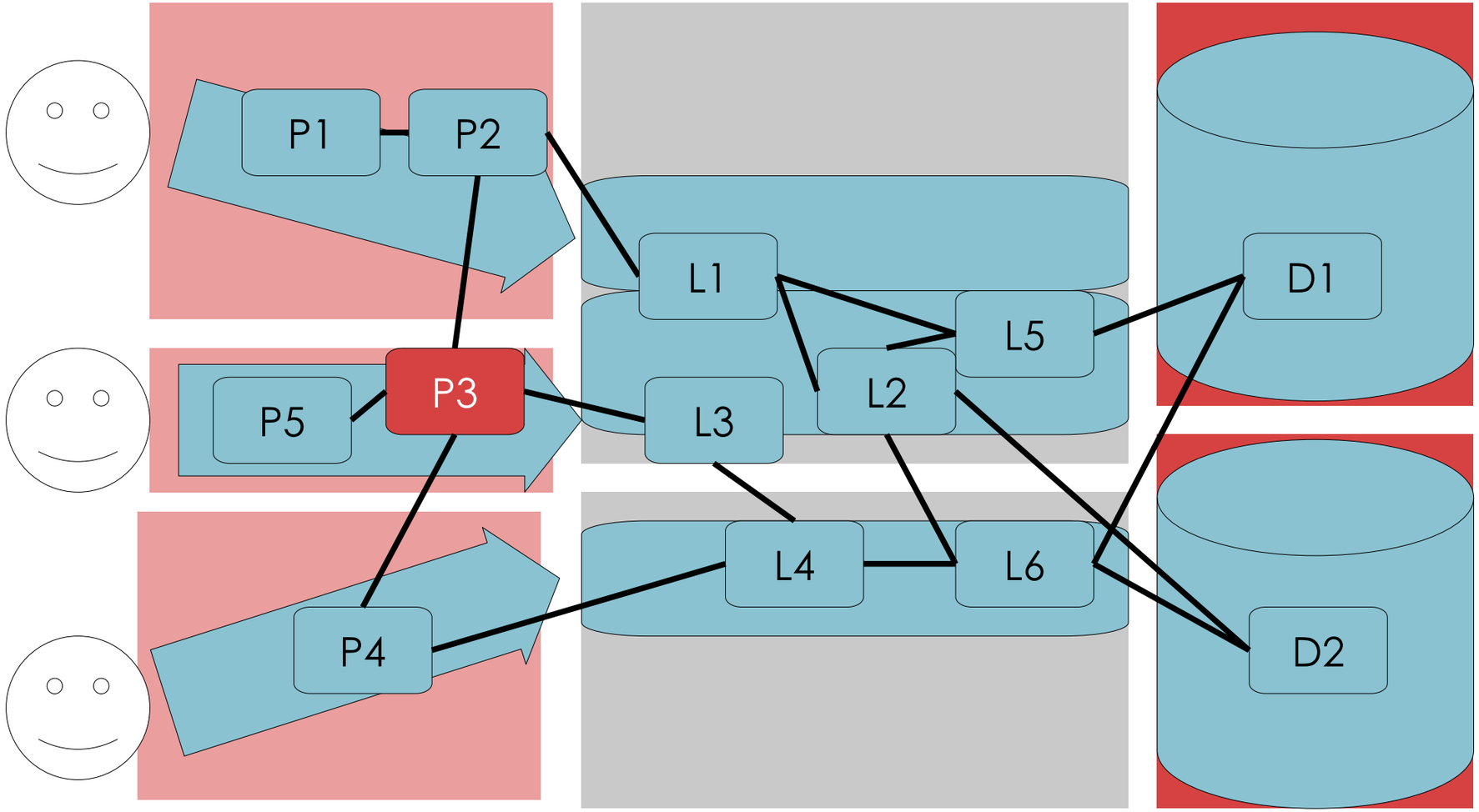


- Verlagerung von Komponenten innerhalb des verteilten Systems
- Dienstbenutzer nimmt Verlagerung nicht wahr

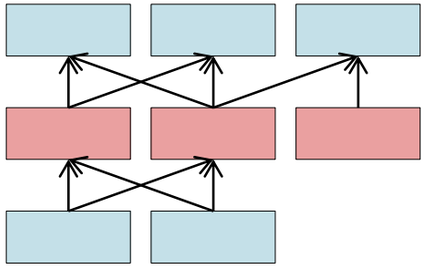
MIGRATIONSTRANSPARENZ



MIGRATIONSTRANSPARENZ

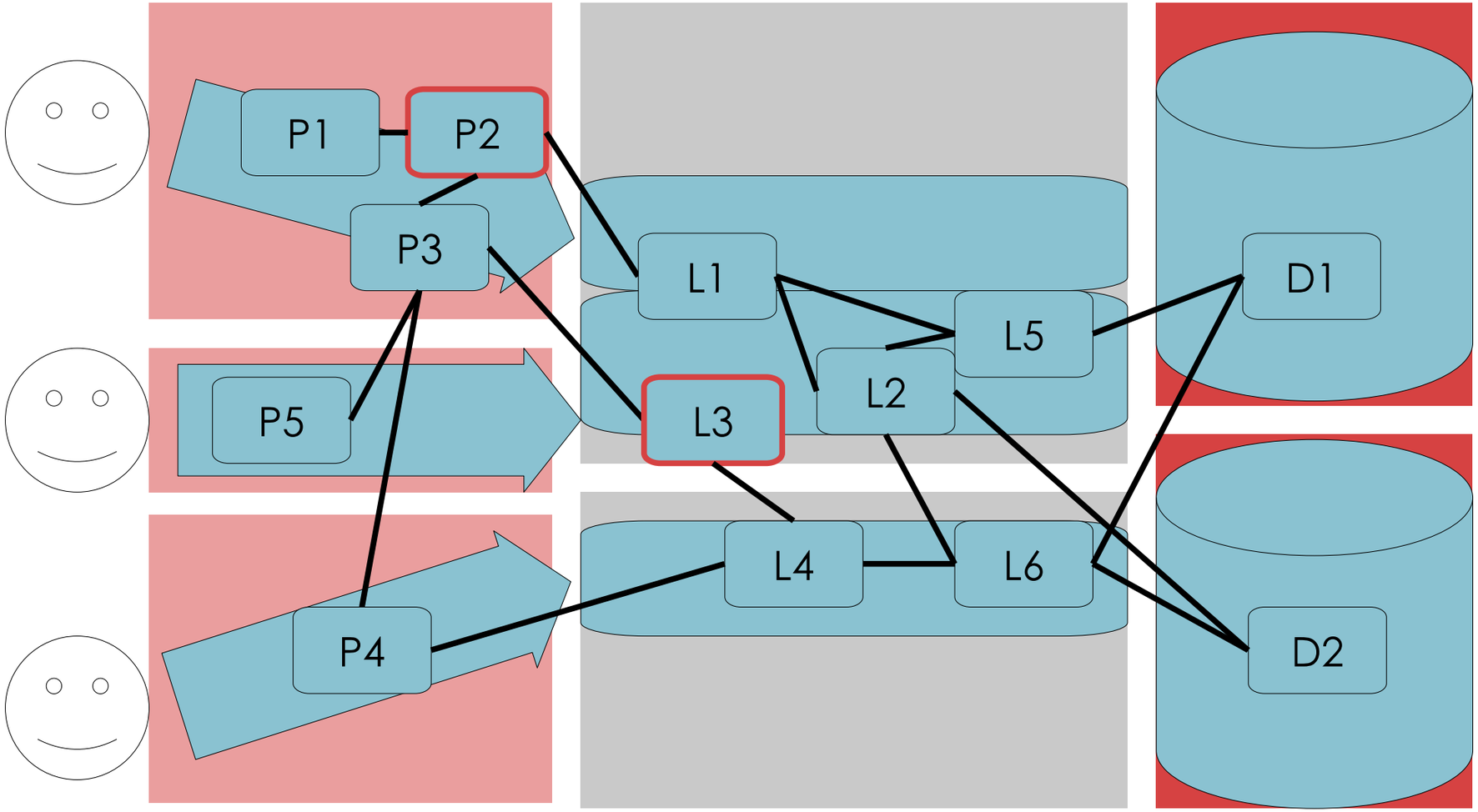


REPLIKATIONSTRANSPARENZ

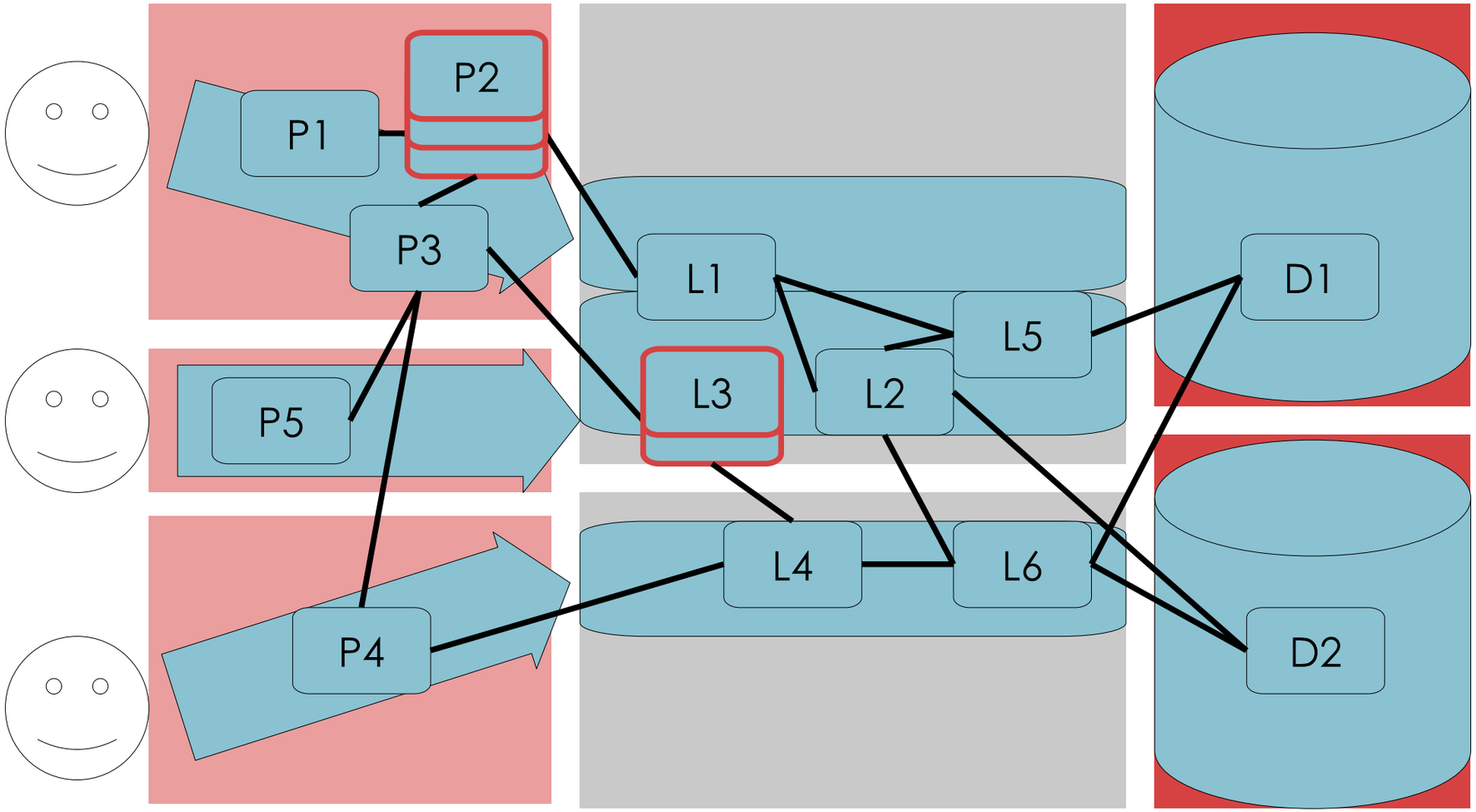


- Mehrere Kopien der gleichen Anwendungskomponente im System
- Im Bedarfsfall (Performanz, Fehlertoleranz) erzeugt oder vernichtet
- Änderung einer Komponente wird für alle übernommen

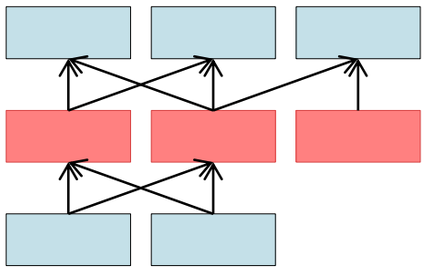
REPLIKATIONSTRANSPARENZ



REPLIKATIONSTRANSPARENZ

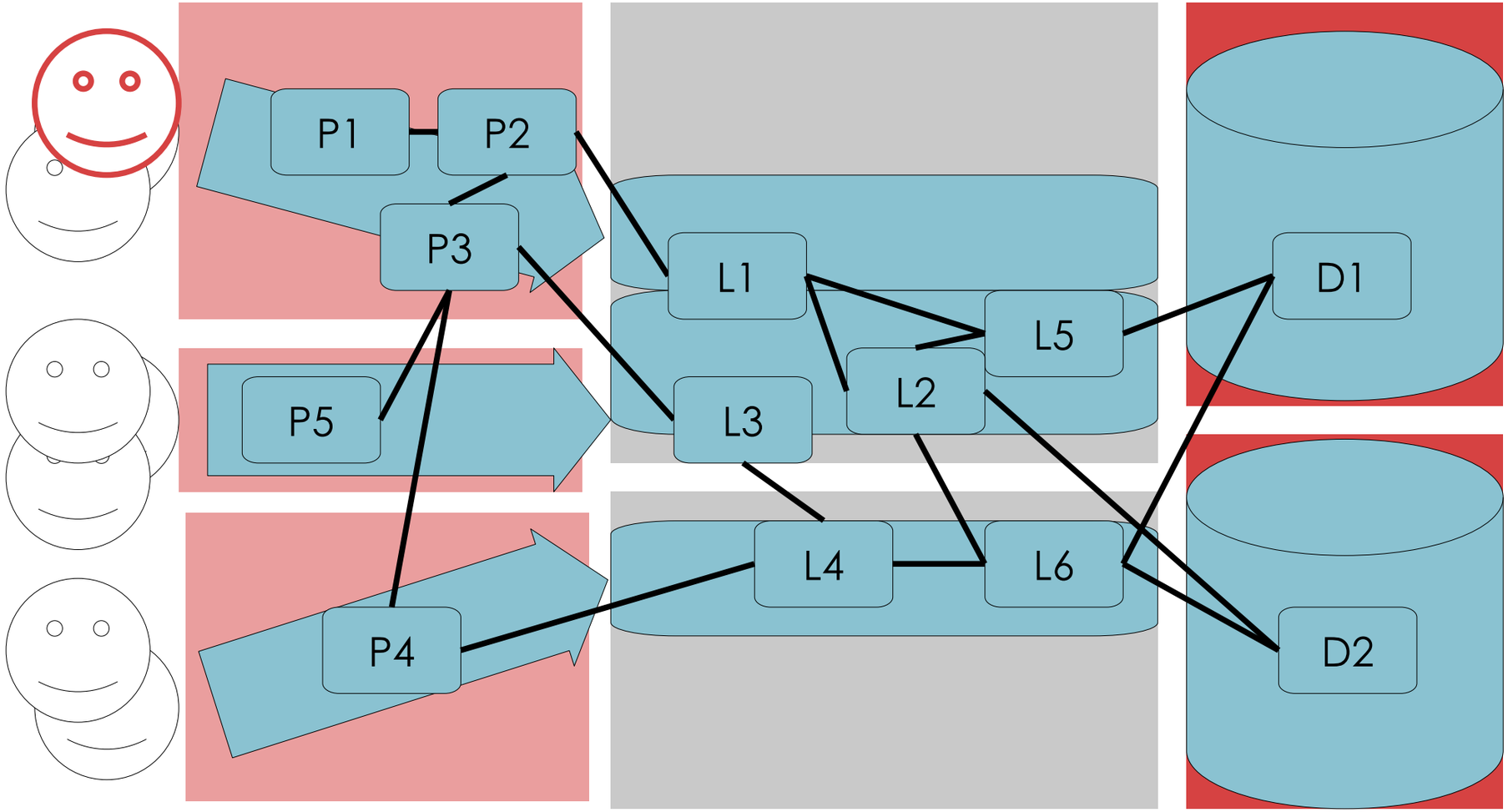


NEBENLÄUFIGKEITSTRANSPARENZ

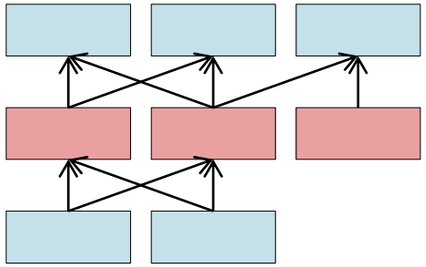


- Gleichzeitiger Zugriff auf Ressourcen erfolgt transparent
- Logische Ein-Nutzer-Sicht
- Performanzerhöhung ebenfalls transparent

NEBENLÄUFIGKEITSTRANSPARENZ



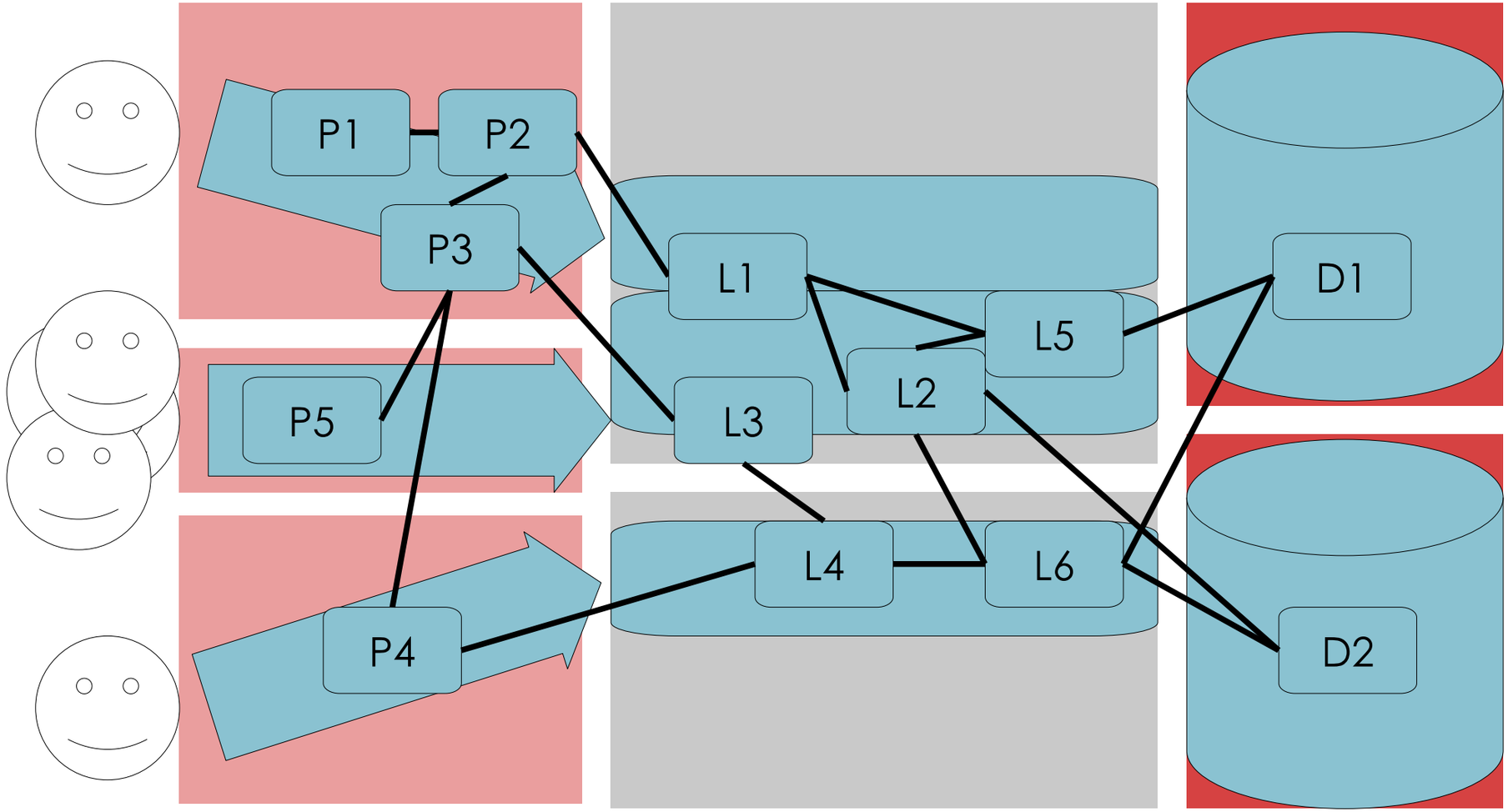
BASIS FÜR „HÖHERE“ NFA



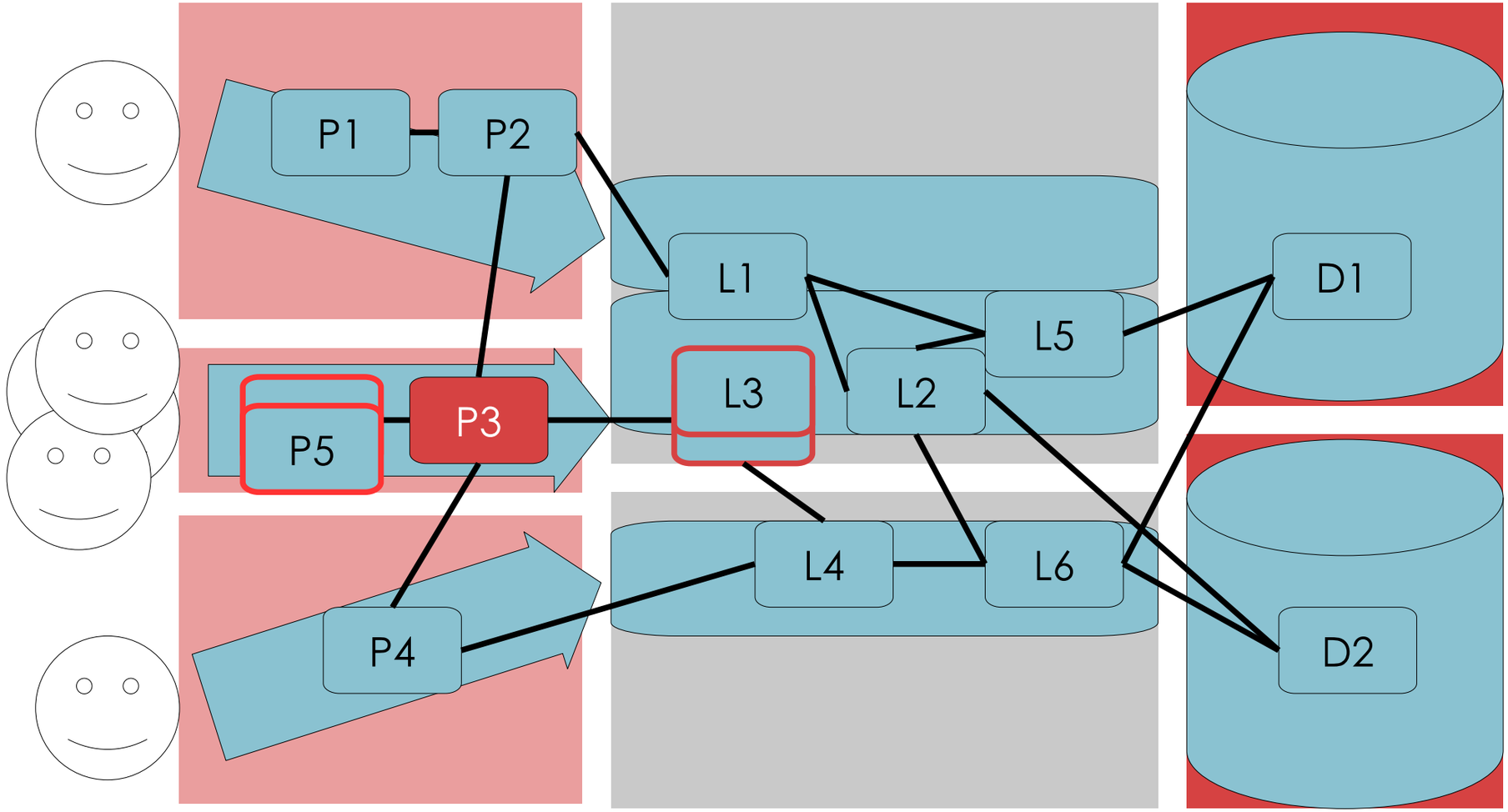
Lastverteilung (engl. Load Balancing)

- Parallelverarbeitung durch Komponentenkopien auf mehreren Rechnern
- Synchronisation der Änderungen nötig
- Gefahr von verlorenen Aktualisierungen und inkonsistenten Analysen

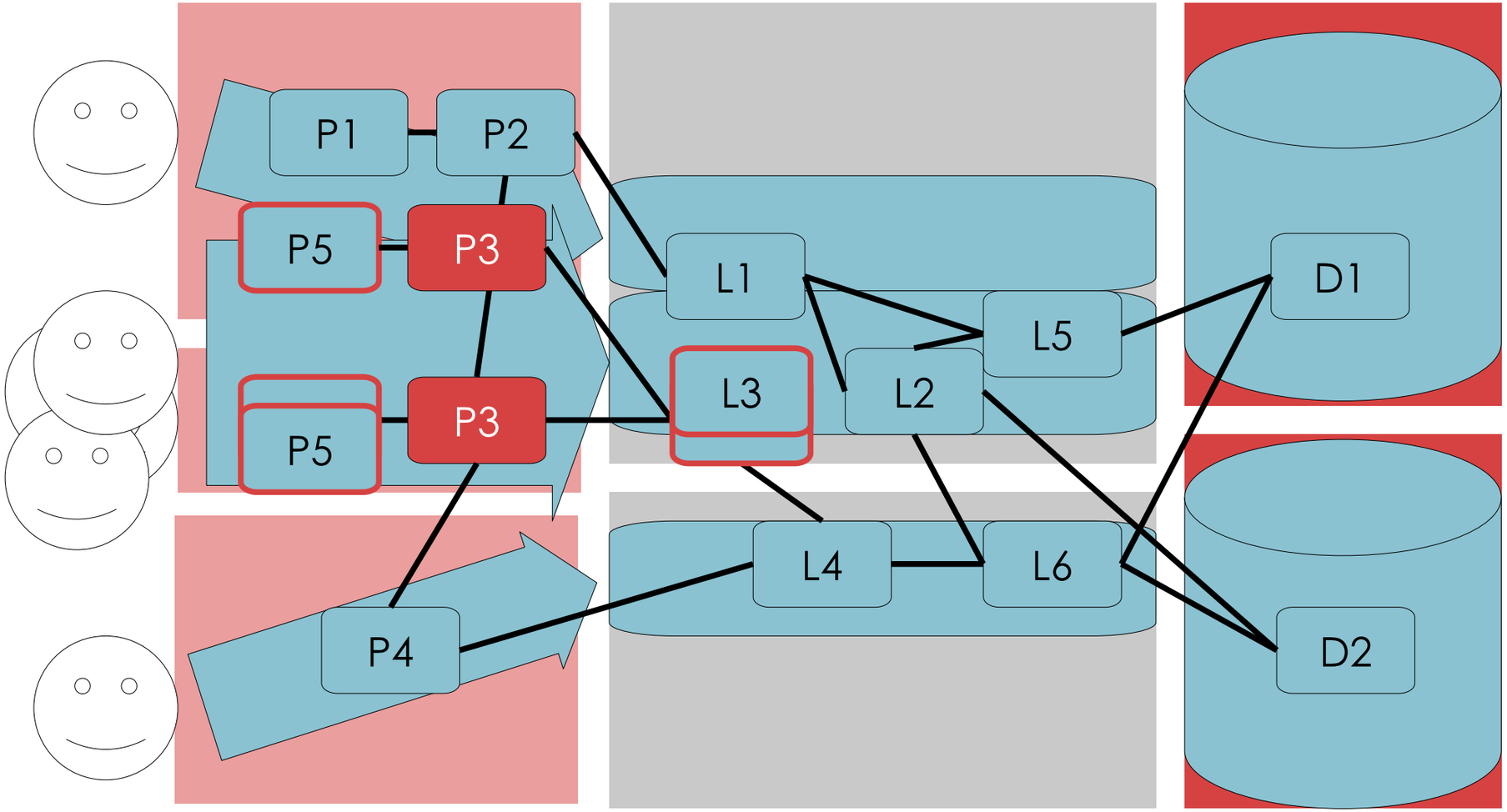
LASTVERTEILUNG

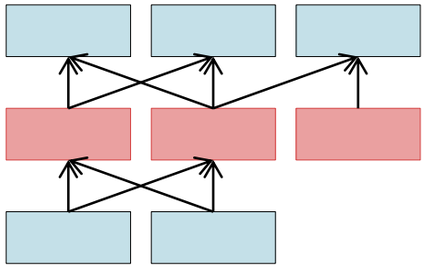


LASTVERTEILUNG



LASTVERTEILUNG





BASIS FÜR „HÖHERE“ NFA

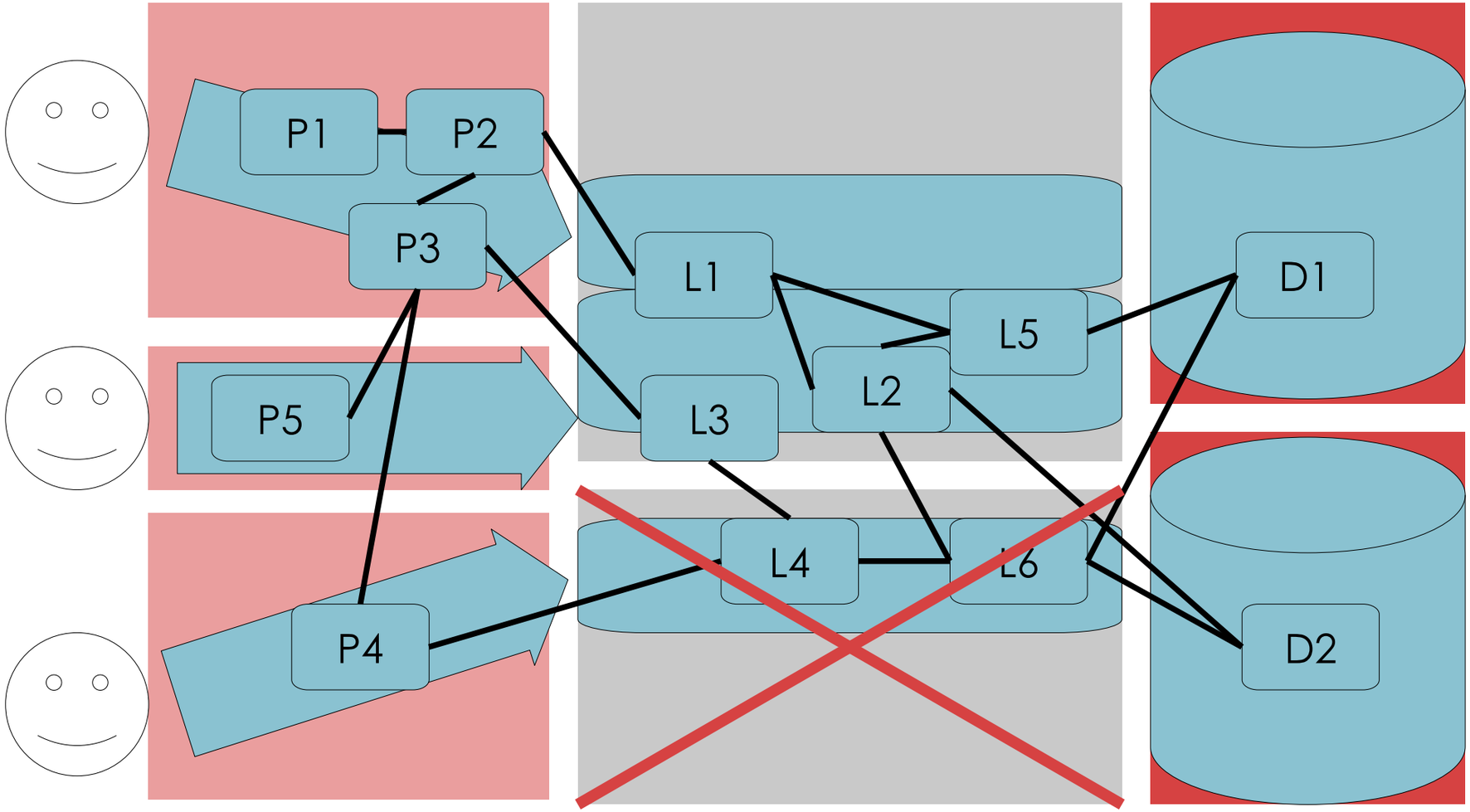
Lastverteilung (engl. *load balancing*)

- Parallelverarbeitung durch Komponentenkopien auf mehreren Rechnern
- Synchronisation der Änderungen nötig
- Gefahr von verlorenen Aktualisierungen und inkonsistenten Analysen

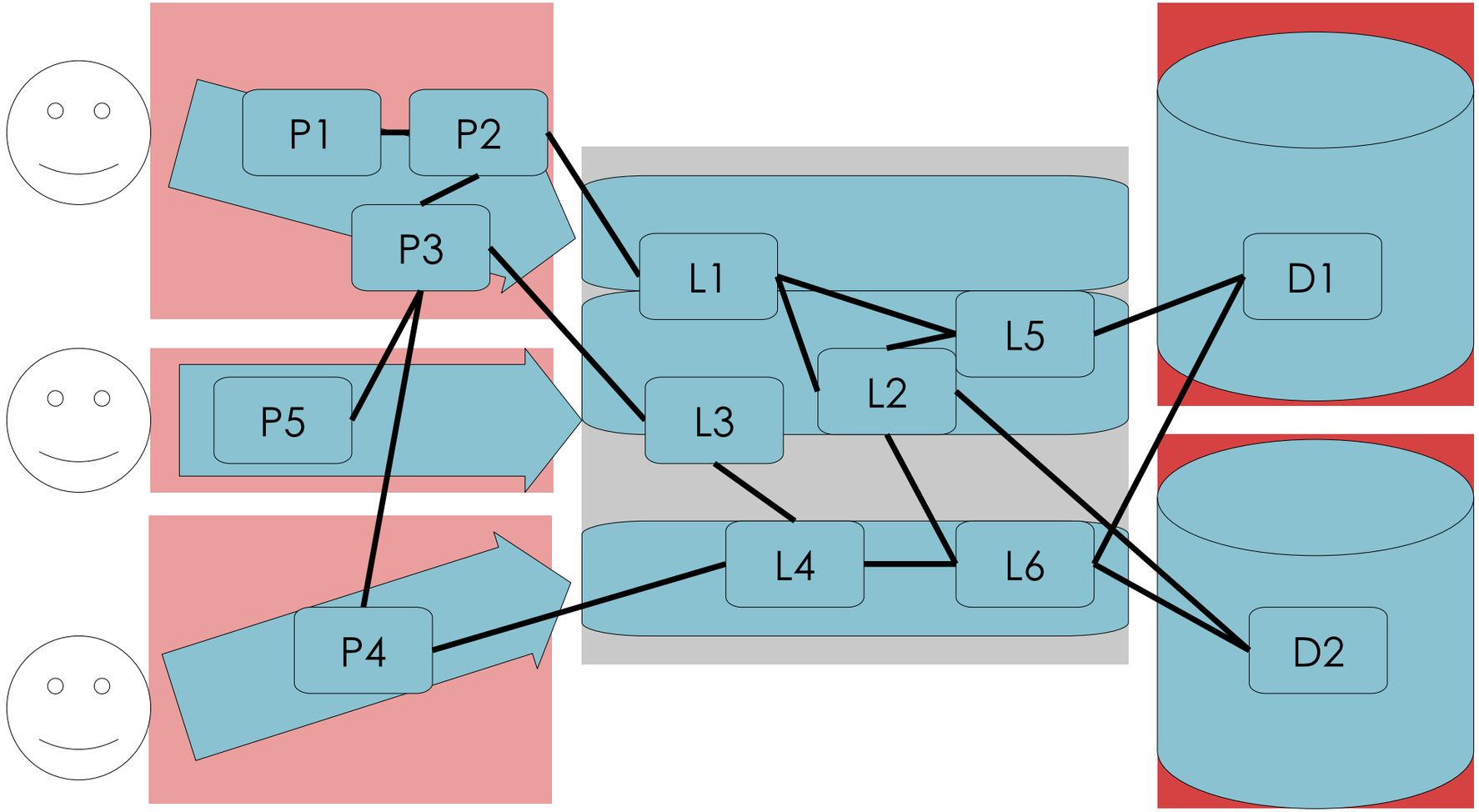
Aufgabenübernahme

- Aufgabenübernahme von ausgefallenen Rechnern (engl. *fail over*)
- Bei schon vorhandenen Kopien kein kritisches Problem
- Ggf. neue Kopie
- Aktivierung einer Sicherheitskopie auf anderem Rechner
- Erzeugen einer komplett neuen Instanz als Kopie der ausgefallenen

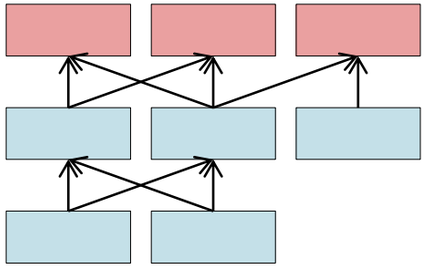
AUFGABENÜBERNAHME



AUFGABENÜBERNAHME



TRANSPARENZDIMENSIONEN „HÖHERER“ NFA



- Skalierbarkeitstransparenz
  - Automatische Anpassung der Verarbeitungskapazität
  - Erweiterung und Austausch von Komponenten

- Performanztransparenz
  - Automatische Lastverteilung zur bestmöglichen Ressourcennutzung
  - Optimale Effizient des Gesamtsystems hinsichtlich Reaktionszeiten

- Fehlertransparenz
  - Ausfall von Teilen des Systems ohne Funktionsbeeinträchtigung
  - Idealerweise unterbrechungsfrei und ohne Performanzverluste

ÜBERSICHT

- Einführung
- Motivation und Herausforderungen
- Beispiel TicketShop
- Architektur
- Transparenz
- Middleware

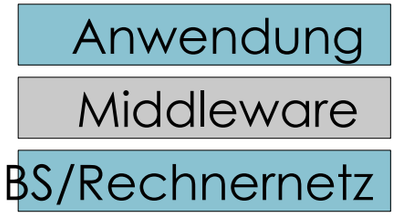
REALISIERUNG VERTEILTER KOMMUNIKATION

- Kommunikation per *Sockets* auf sehr niedrigen Abstraktionsniveau
- Eigene Protokolle für Kommunikation erzeugen Zusatzaufwand
  - Ver-/Entpackung der Byteketten für Daten, -typen und Sprachkonstrukte
  - Fallunterscheidung des zu nutzenden Dienstes
  - Aufrufe zum Senden und Empfangen der Daten
  - Überprüfung der Typsicherheit
  - Zusätzlicher überwiegend technischer Aufwand
  - Teilweise API-Untersützung beispielsweise bei Objektserialisierung

LÖSUNGSANSATZ

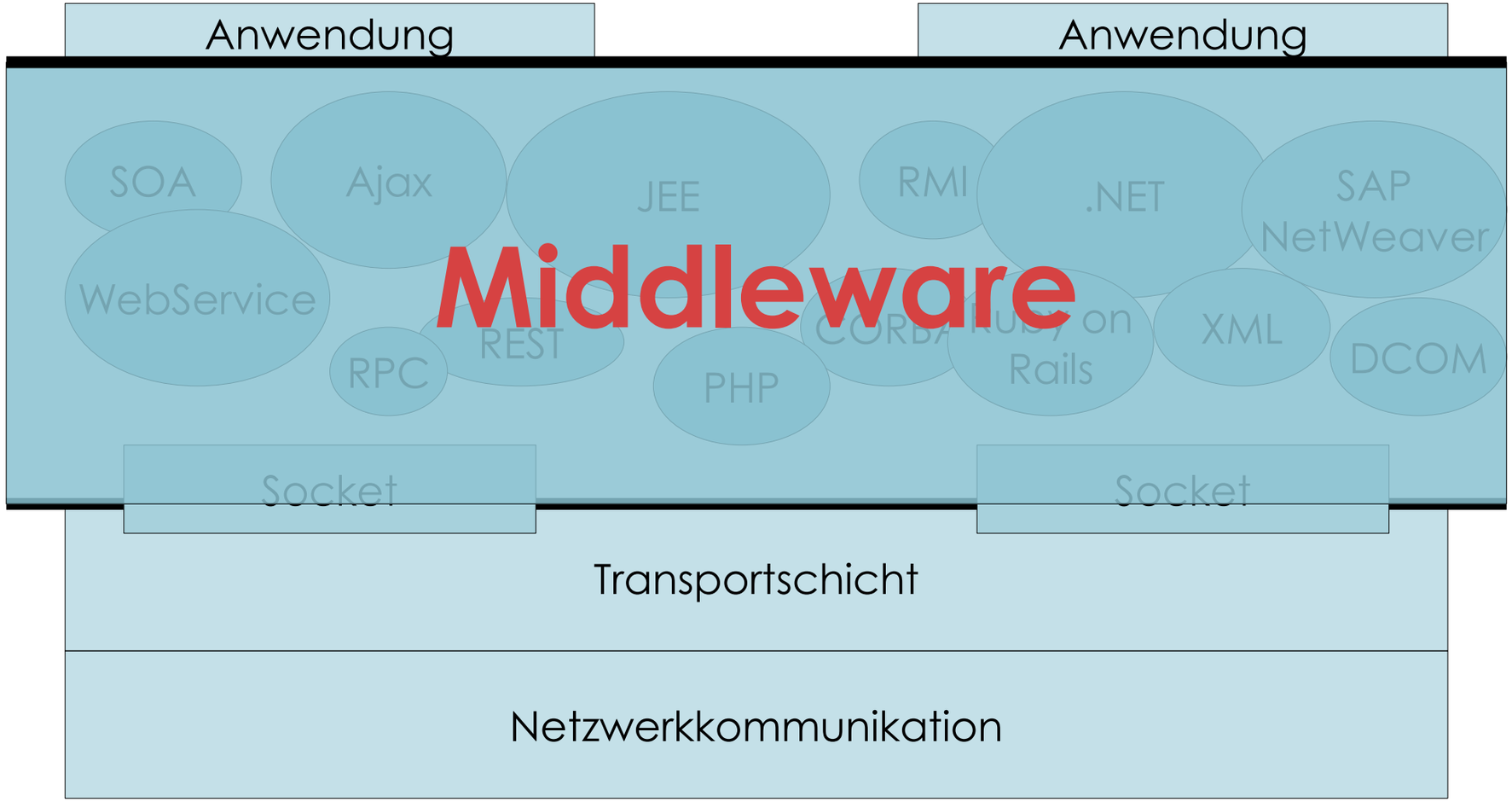
- Abstraktionsebene der Kommunikation erhöhen
- Abstrahieren von technischen Details der verteilten Kommunikation
- Konzentration auf fachliche Anforderungen
- Einführung einer anwendungsneutralen Abstraktionsebene
  - Übernimmt/Realisiert technische Aspekte der verteilten Kommunikation
  - Serialisieren, Senden, Empfangen, Deserialisieren
  - Identifikation, Aufrufweiterleitung, Fehlerbehandlung
- Realisiert Transparenz für den Nutzer und den Entwickler!

MIDDLEWARE

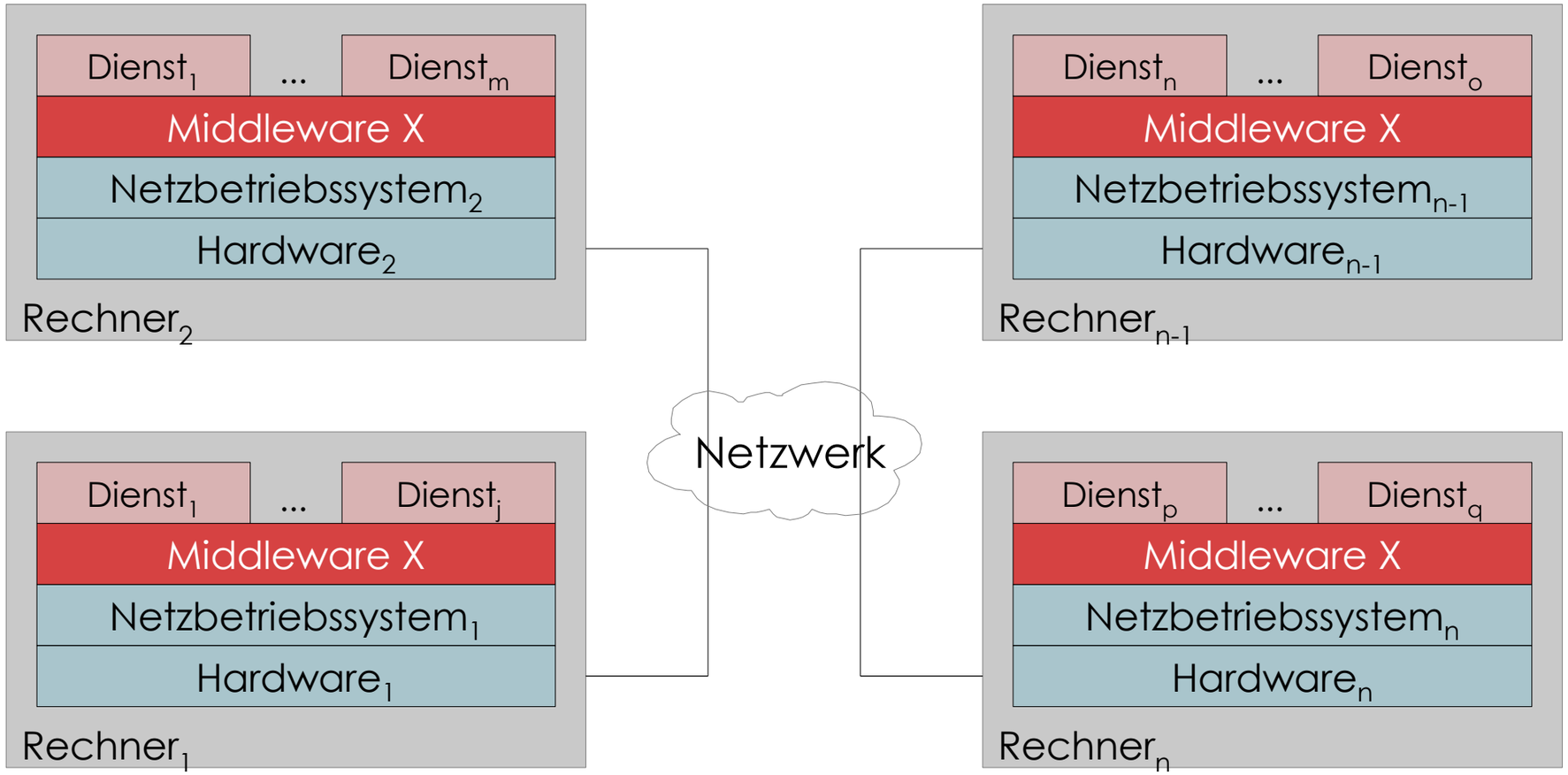


- „Zwischenanwendung“ oder anwendungsneutrale Programme
- Vermitteln zwischen Anwendung und Infrastruktur
- Komplexität des Gesamtsystem und der Infrastruktur bleibt verborgen
- Eine Ebene in einem komplexen Softwaresystem
- Implementiert die benötigten Transparenzdimensionen
- Realisiert somit die NFA (Skalierbarkeit, Wartbarkeit, Performanz,...)
- Ermöglicht entkoppelten Softwarekomponenten den Datenaustausch

MIDDLEWARE


Anwendung
Anwendung
SOA
Ajax
JEE
RMI
.NET
SAP NetWeaver
WebService
Middleware
REST
RPC
PHP
CORBA
Ruby on Rails
XML
DCOM
Socket
Socket
Transportschicht
Netzwerkkommunikation

MODELL EINES VERTEILTEN SYSTEMS



Ähnlich zu: Emmerich, W.: Konstruktion von verteilten Objekten, S. 9

UNIVERSITÄT LEIPZIG

VIELEN DANK!

Pascal Kovacs

Institut für Wirtschaftsinformatik

pkovacs@uni-leipzig.de

https://www.wifa.uni-leipzig.de/iwi/